Instituto Tecnológico de Costa Rica
Escuela de Ingeniería en Computadores
– Bases de Datos
II Semestre 2026

SmartCity Analytics

Arquitectura de datos urbanos con PostgreSQL, MongoDB, Apache Hadoop y Apache Spark

---

Documento técnico del Proyecto

Profesor: Marco Rivera

Integrante: Joseph Murillo Álvarez

Fecha de entrega: 28 de Octubre de 2026

Índice

1. Base de datos relacional: PostgreSQL **1**
1.1. Rol de PostgreSQL en la arquitectura . . . 1
1.2. Descripción del problema y fuentes de datos . . . 2
1.3. Flujo de ingestión . . . 3
1.4. Modelo relacional . . . 5
1.4.1. Entidades y diagrama entidad-relación . . . 5
1.4.2. Llaves y restricciones . . . 8
1.4.3. Normalización hasta tercera forma normal . . . 8
1.4.4. Decisiones de diseño y limitaciones . . . 11
1.5. Índices . . . 12
1.6. Particionamiento . . . 12
1.7. Vista materializada y función almacenada . . . 14
1.8. Consultas obligatorias . . . 15
1.9. Consultas avanzadas . . . 17
1.9.1. Expresión de tabla común (CTE) . . . 17
1.9.2. Funciones de ventana . . . 17
1.9.3. Consultas sobre fechas . . . 19
1.9.4. Preguntas de ejemplo del enunciado . . . 20
1.10. Resultados . . . 21
1.11. Evaluación del rendimiento: índices y particionamiento . . . 22
1.12. Evaluación: ventajas, limitaciones y escalabilidad . . . 24
1.13. Conclusiones de la parte relacional . . . 25

2. Base de datos documental: MongoDB **26**
2.1. Rol de MongoDB en la arquitectura . . . 26
2.2. Fuentes de datos y volumen . . . 27
2.3. Flujo de ingestión y generación de datos . . . 28
2.3.1. Datos derivados y sintéticos . . . 29
2.4. Modelo documental . . . 30
2.5. Índices . . . 33
2.6. Operaciones CRUD . . . 34
2.7. Consultas de agregación y resultados . . . 34
2.7.1. Alertas por zona . . . 36
2.7.2. Patrón horario: tráfico y contaminación . . . 37
2.7.3. Distribución de lecturas y eventos . . . 38
2.7.4. Incidentes . . . 38

2.8. Evaluación: ventajas, limitaciones y escalabilidad 39

A. Anexo A: Evidencia de PostgreSQL 41

B. Anexo B: Evidencia de MongoDB 47

C. Anexo C: Evidencia de Datos de Origen 50

Índice de figuras

1. Flujo de ingestión y preparación de datos hacia PostgreSQL. . 3
2. Modelo entidad-relación de PostgreSQL. Cada flecha une una llave primaria (lado 1) con su llave foránea (lado N). 7
3. Corrección de la relación entre organismo y sensor. 9
4. Filas por partición antes y después de crear las particiones 2016–2020. 13
5. Promedio diario y promedio móvil de 7 días del PM2.5 en la zona Benito Juárez. Origen de los datos: PostgreSQL. 18
6. Resultados de consultas en PostgreSQL: sensores por organismo, mediciones por año, vehículos por zona y PM2.5 por zona. Origen de los datos: PostgreSQL. 21
7. Efecto del índice sobre `pm25`. El tiempo baja de 14.3 segundos a 4.4 segundos con el índice. Origen de los datos: PostgreSQL (`EXPLAIN ANALYZE`). 23
8. Cobertura temporal de las dos fuentes. Origen de los datos: MongoDB. 28
9. Flujo de ingestión desde los archivos CSV hasta las colecciones de MongoDB. 29
10. Modelo documental de la base `smartcity`. 31
11. Alertas por zona geográfica. Origen de los datos: MongoDB, colección `alertas`. 36
12. Vehículos y PM2.5 promedio por hora del día. Origen de los datos: MongoDB, colección `lecturas`. 37
13. Lecturas por tipo de sensor y eventos por tipo. Origen de los datos: MongoDB, colecciones `lecturas` y `eventos`. 38
14. Incidentes por tipo y por zona. Origen de los datos: MongoDB, colección `incidentes`. 38
15. Conteo total de registros por tabla en PostgreSQL (1,062,858 en `medicion`). 41
16. Distribución de filas por año en la tabla particionada `medicion`. 42
17. Diagrama Entidad-Relación generado en DBeaver, incluyendo la tabla `resultado_spark_zonas`. 43
18. Definición de llaves primarias, foráneas e índices en PostgreSQL. 44
19. `EXPLAIN ANALYZE` sin índice (Parallel Seq Scan), mostrando un tiempo de 14.3 segundos. 45

20. `EXPLAIN ANALYZE` con índice (Index Only Scan), reduciendo el tiempo a 4.4 segundos. 46
21. `EXPLAIN ANALYZE` con *Partition Pruning* para enero de 2024, reduciendo el escaneo a una sola partición. 47
22. Colecciones en MongoDB y cantidad de documentos (500,000 lecturas, 79,428 alertas). 47
23. Aggregation Framework calculando el promedio de PM2.5 por zona. 48
24. Plan de ejecución usando `IXSCAN` sobre el índice de `timestamp`. 49
25. Inspección del CSV de tráfico (204,000 filas, 47 columnas). . . 50
26. Inspección del CSV de calidad del aire (858,870 filas, 5 contaminantes). 50

Índice de cuadros

1. Fuentes de datos almacenadas en PostgreSQL. 2
2. Archivos SQL de PostgreSQL. 5
3. Entidades mínimas del enunciado y su implementación. 6
4. Llaves primarias, foráneas y restricciones. 8
5. Decisiones de diseño, justificación y limitaciones. 11
6. Índices creados en PostgreSQL. 12
7. Filas y tamaño por partición de `medicion`. 13
8. Problemas detectados al particionar `medicion` y su corrección. 14
9. Consultas obligatorias de PostgreSQL y resultados. 16
10. Resultado de `LAG` para la zona Benito Juárez (2021). 19
11. Preguntas de ejemplo resueltas con PostgreSQL. 20
12. Resultados de `EXPLAIN ANALYZE` (tiempos medidos en la base real del proyecto). 22
13. Valoración de PostgreSQL según la necesidad, con evidencia del proyecto. 24
14. Fuentes de datos almacenadas en la colección `lecturas`. . . . 27
15. Origen de cada colección y campo derivado. 30
16. Índices creados en MongoDB. 33
17. Operaciones CRUD demostradas. 34
18. Consultas obligatorias de MongoDB y resultados. 35
19. Valoración de MongoDB según la necesidad, con evidencia del proyecto. 39

1. Base de datos relacional: PostgreSQL

1.1. Rol de PostgreSQL en la arquitectura

PostgreSQL almacena la información estructurada y relacional de SmartCity: los datos maestros (sensores, ubicaciones, categorías, organismos responsables y usuarios), las mediciones ya limpias de todos los sensores y los resultados agregados de alta calidad que produce Spark (tabla `resultado_spark_zonas`). MongoDB guarda las lecturas con estructura variable y los eventos (sección 2); PostgreSQL guarda lo que debe ser consistente, relacionable y consultable con SQL.

La elección de un modelo relacional se justifica por cinco características de estos datos:

- **Estructura estable y conocida.** Un sensor siempre tiene nombre, tipo, estado, zona, categoría y organismo; una medición siempre pertenece a un sensor y tiene una fecha. El esquema se define una sola vez y el motor lo hace cumplir.
- **Relaciones entre entidades.** Cada sensor se ubica en una zona, pertenece a una categoría y depende de un organismo. Las preguntas del negocio ("sensores por organismo", "zonas con más sensores") son uniones entre estas entidades, que SQL resuelve con `JOIN`.
- **Integridad referencial.** Las llaves foráneas impiden registrar una medición de un sensor que no existe o asignar un sensor a una zona inexistente. MongoDB no ofrece esta garantía.
- **Analítica con SQL avanzado.** Expresiones de tabla comunes (CTE), funciones de ventana, subconsultas y vistas materializadas permiten responder preguntas complejas sin salir del motor.
- **Volumen manejable con particionamiento.** Con 1.062.858 mediciones ordenadas en el tiempo, el particionamiento por rango de fecha y los índices mantienen las consultas en milisegundos (sección 1.11).

PostgreSQL no es la mejor opción para datos cuya forma cambia de un sensor a otro, ni para repartir un volumen muy superior entre varios servidores; esas necesidades quedan para MongoDB y para HDFS con Spark (sección 1.12).

1.2. Descripción del problema y fuentes de datos

La municipalidad de SmartCity ha desplegado cientos de sensores que generan información de movilidad y de calidad del aire. Necesita responder preguntas como: ¿qué zona registra más vehículos?, ¿cuál presenta mayor contaminación?, ¿en qué hora hay más congestión?, ¿qué sensores dejaron de enviar información? y ¿cómo se distribuyen los sensores entre los organismos responsables?

Se integraron dos fuentes de datos distintas, como exige el enunciado. La tabla 1 resume su contenido tal como quedó almacenado en la tabla `medicion`.

Cuadro 1: Fuentes de datos almacenadas en PostgreSQL.

Fuente	**Contenido**	**Periodo**	**Mediciones**	**Sensores**
Tráfico vehicular (`smart_city_traffic_mobility.csv`)	Vehículos, congestión, temperatura, AQI, emisiones y accidentes de 100 intersecciones (`INT_xxx`) en 6 zonas; una lectura por hora	1 ene – 26 mar 2026 (85 días)	204.000	100
Calidad del aire (OpenAQ)	PM2.5, PM10, NO_2, O_3 y CO de estaciones en un radio de 25 km del centro de la Ciudad de México (`CDMX_xxx`); datos horarios	10 mar 2016 – 21 jun 2026	858.858	184
Total			**1.062.858**	**284**

Fuentes de los datos. El conjunto de tráfico vehicular proviene de Kaggle (*Smart City Traffic & Urban Mobility Dataset*, de Mobeen Fatima, publicado en agosto de 2026, licencia CC BY 4.0, 204.000 filas). El conjunto de calidad del aire se obtuvo mediante la API pública OpenAQ v3 (openaq.org), consultando estaciones en un radio de 25 km del centro de la Ciudad de México. Ambos conjuntos son de acceso público y se utilizaron únicamente con fines académicos. La descarga se realizó en octubre de 2026.

https://www.kaggle.com/datasets/mobeenfatimah/cityflow-smart-urban-mobility-and-traffic-iot/data

A estos 284 sensores con datos se suma un sensor de prueba (`S-TEST-INACTIVO`, estado `inactivo`, sin mediciones), creado a propósito para demostrar las consultas de sensores inactivos y sensores sin actividad. En total hay 285 sensores en 38 zonas: 6 zonas ficticias de tráfico (`SmartCity District`) y 32 zonas de la Ciudad de México (`CDMX`).

El volumen cumple el escenario mínimo del enunciado (100.000 registros en PostgreSQL): hay 1.062.858 mediciones, 10,6 veces el mínimo. MongoDB almacena un subconjunto de 500.000 lecturas del mismo conjunto limpio, por lo que algunos resultados numéricos difieren entre ambos motores. Por ejemplo, la zona con mayor PM2.5 promedio es *Mexico City* en PostgreSQL (todo el histórico) y *Merced* en MongoDB (subconjunto).

Las dos fuentes no comparten zonas ni cobertura. El tráfico usa zonas ficticias (*Financial District*, *Downtown Core*, etc.) y cubre apenas 85 días de 2026, mientras que el aire usa zonas de la Ciudad de México y abarca diez años con una cobertura muy irregular: 2022 tiene 2.142 mediciones y septiembre de 2024 solo 54, frente a 180.774 en 2021. Por eso las fuentes no se cruzan por zona; se comparan por patrón horario (sección 1.10) y, en Spark, por sus agregados.

1.3. Flujo de ingestión

La figura 1 resume el flujo hasta PostgreSQL. Los pasos son:

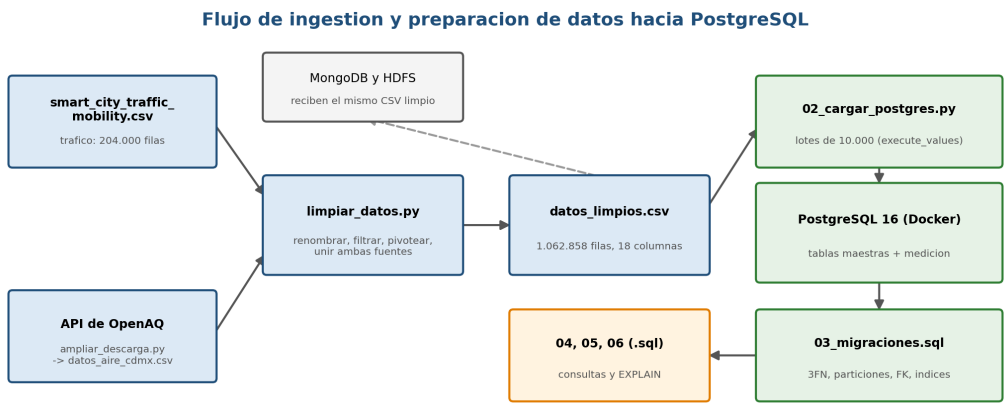


Figura 1: Flujo de ingestión y preparación de datos hacia PostgreSQL.

1. **Descarga** (`ampliar_descarga.py`). Se consultan las estaciones de OpenAQ en un radio de 25 km del centro de la Ciudad de México y, para cada sensor de PM2.5, PM10, NO_2, O_3 y CO, se descargan hasta cinco páginas de 1.000 mediciones horarias.

2. **Limpieza** (`limpiar_datos.py`). Se renombran las columnas al esquema común, se eliminan las filas con fecha, zona o sensor nulos y los vehículos negativos, y se *pivotea* el aire (el original trae una fila por contaminante; se transforma a una fila por sensor y fecha con una columna por contaminante). Ambas fuentes se unen en `datos_limpios.csv`: 1.062.858 filas y 18 columnas.

3. **Carga inicial** (`02_cargar_postgres.py`). Inserta la ciudad, las zonas, las categorías, los sensores, los organismos y las mediciones, estas últimas con `execute_values` en lotes de 10.000 filas. Las fechas de formato mixto se unifican a UTC y se almacenan como `TIMESTAMP` sin zona horaria, y los indicadores 0/1 se convierten a booleanos.

4. **Evolución del esquema** (`03_migraciones.sql`). Agrega la tabla de usuarios, la fecha de instalación, el distrito, el particionamiento, la corrección de organismos y las llaves foráneas (secciones 1.4 y 1.6).

5. **Verificación.** El script `verificar.ps1` crea una copia de la base, reconstruye el esquema desde cero con `01_ddl.sql` en una base vacía y ejecuta los archivos `02` a `06`; los seis terminaron sin errores.

Reproducibilidad. `01_ddl.sql` crea el esquema final en una base vacía (verificado), pero el cargador `02_cargar_postgres.py` corresponde al esquema inicial. Los datos se cargaron con ese esquema y el esquema final se alcanzó con `03_migraciones.sql`; por eso la carga de datos no se puede repetir directamente sobre `01_ddl.sql`.

La tabla 2 lista los archivos SQL entregados.

Cuadro 2: Archivos SQL de PostgreSQL.

Archivo	**Contenido**
`01_ddl.sql`	Esquema final: tablas, restricciones, particiones, índices, vista materializada y función. Se ejecuta en una base vacía
`02_datos_referencia.sql`	Ciudad, categorías, organismos, usuarios y sensor de prueba (idempotente)
`03_migraciones.sql`	Historial y pasos para llevar la base inicial al esquema final (repetible)
`04_consultas_obligatorias.sql`	Las diez consultas obligatorias
`05_consultas_avanzadas.sql`	Consultas básicas, CTE, funciones de ventana, fechas y preguntas de ejemplo
`06_rendimiento.sql`	`EXPLAIN ANALYZE`, índices y *partition pruning*

1.4. Modelo relacional

1.4.1. Entidades y diagrama entidad-relación

La tabla 3 relaciona las entidades mínimas del enunciado con las tablas implementadas, y la figura 2 muestra el modelo completo.

Cuadro 3: Entidades mínimas del enunciado y su implementación.

Entidad	**Tabla**	**Atributos requeridos**	**Adicionales**
Sensores	`sensor`	`id_sensor`, `nombre`, `tipo_sensor`, `fecha_instalacion`, `estado`	`id_zona`, `id_categoria`, `id_organismo` (FK)
Ubicaciones	`zona`	`id_zona`, `nombre_zona`, `distrito`, coordenadas (`latitud`, `longitud`)	`ciudad_origen`, `id_ciudad` (FK)
Categorías de sensores	`categoria_sensor`	`id_categoria`, `nombre_categoria`	–
Usuarios del sistema	`usuarios`	`id_usuario`, `nombre`, `correo`	–
Organismos responsables	`organismo`	`id_organismo`, `nombre`, `responsable`	–
(adicional)	`ciudad`	`id_ciudad`, `nombre`	Agrupa las zonas
(adicional)	`medicion`	`id_medicion`, `id_sensor`, `fecha` y 13 variables medidas	Tabla de hechos particionada

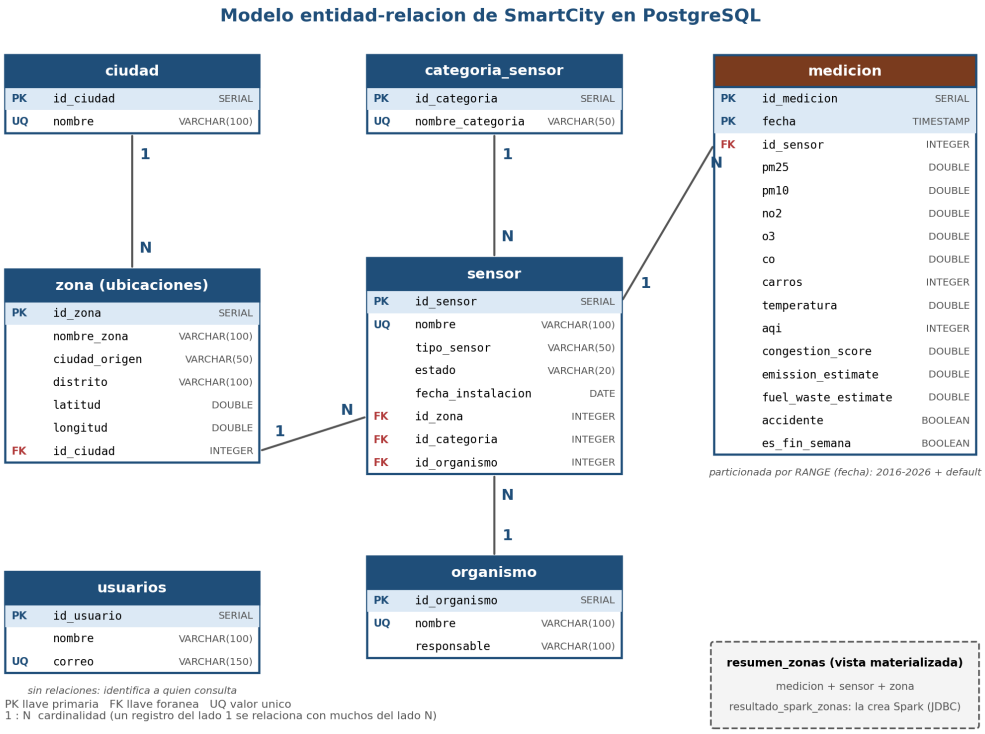


Figura 2: Modelo entidad-relación de PostgreSQL. Cada flecha une una llave primaria (lado 1) con su llave foránea (lado N).

1.4.2. Llaves y restricciones

Cuadro 4: Llaves primarias, foráneas y restricciones.

Tabla	Llave primaria	Llaves foráneas	Otras
ciudad	id_ciudad	–	nombre único
zona	id_zona	id_ciudad → ciudad	nombre_zona no nulo
categoria_sensor	id_categoria	–	nombre_categoria único
organismo	id_organismo	–	nombre único
sensor	id_sensor	id_zona, id_categoria, id_organismo	nombre único; estado por defecto activo
usuarios	id_usuario	–	correo único
medicion	(id_medicion, fecha)	id_sensor → sensor (fk_medicion_sensor)	fecha no nula

1.4.3. Normalización hasta tercera forma normal

- **Primera forma normal (1FN).** Todos los atributos son atómicos: cada columna guarda un solo valor y no hay grupos repetidos. Las coordenadas se guardan en dos columnas (latitud y longitud), no como texto compuesto.
- **Segunda forma normal (2FN).** Las tablas maestras tienen llave primaria simple, por lo que no puede haber dependencias parciales. En medicion la llave es compuesta (id_medicion, fecha), pero no por una necesidad lógica: PostgreSQL exige que la llave primaria de una tabla particionada incluya la columna de partición. Como id_medicion es único por sí solo, formalmente los atributos dependen de una parte de la llave; es una desviación consciente y justificada por el particionamiento.
- **Tercera forma normal (3FN).** Ningún atributo no clave depende de otro atributo no clave. Los datos de la zona (nombre, distrito, coorde-

nadas) viven en `zona`, no en cada medición; los del organismo (nombre, responsable) viven en `organismo`, no en cada sensor.

Corrección aplicada: el organismo no debe depender del sensor. El diseño inicial guardaba `id_sensor` dentro de `organismo`, lo que invertía la cardinalidad: para asignar un mismo organismo a muchos sensores había que repetir su nombre y su responsable, con la redundancia y las anomalías de actualización que eso implica. Se movió la referencia a `sensor.id_organismo` (figura 3). Los 285 sensores quedaron asignados: MINAE 185, MOPT 98, Municipalidad 1 y CNFL 1. La distribución es desigual porque Municipalidad y CNFL conservan la asignación original (un sensor cada uno) y el resto se repartió por categoría: ambiente a MINAE y movilidad a MOPT.

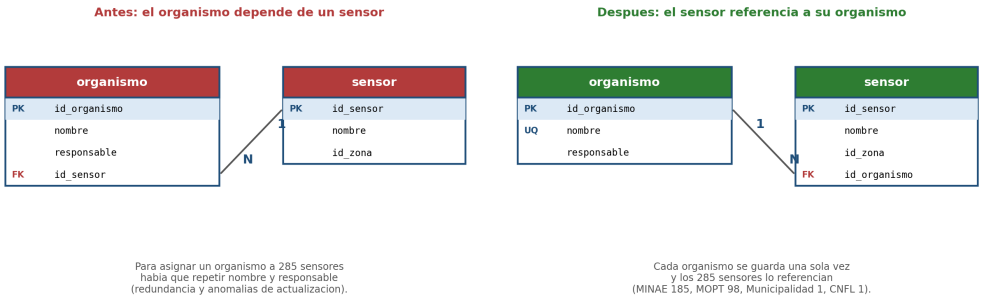


Figura 3: Corrección de la relación entre organismo y sensor.

1.4.4. Decisiones de diseño y limitaciones

Cuadro 5: Decisiones de diseño, justificación y limitaciones.

Decisión	**Justificación**	**Limitación**
Una sola tabla `medicion` para aire y tráfico	Ambas fuentes comparten sensor, fecha y zona, y una sola tabla simplifica las consultas y el particionamiento por fecha	Es una tabla ancha: las columnas de aire quedan en `NULL` en las filas de tráfico, y al revés. Un nuevo tipo de sensor exige `ALTER TABLE`
`zona.distrito` derivado	El enunciado pide el distrito. Se asigna `CDMX` a las zonas con sensores de calidad del aire y `SmartCity District` a las de tráfico	Es una clasificación propia, no un dato de la fuente
`sensor.fecha_instalacion` derivada	Las fuentes no traen la fecha de instalación; se toma la primera medición de cada sensor	Es una aproximación. El sensor de prueba tiene una fecha fija (1 ene 2025)
`sensor.tipo_sensor` junto a la categoría	`tipo_sensor` es más específico (`calidad_aire`, `trafico`, `test`) que la categoría (`ambiente`, `movilidad`) y simplifica los filtros	Está parcialmente repetido con la categoría
`zona.ciudad_origen`	Se conserva del diseño inicial	Todas las zonas tienen el valor `Mixto`, por lo que no aporta información; el distrito lo reemplaza en la práctica
`usuarios` sin relaciones	El enunciado solo exige la entidad. Identifica a quién consulta el sistema; no se pide gestión de usuarios ni permisos	No hay trazabilidad de quién registró cada dato
Estaciones de nombre parecido	*FES Aragon* y *FES Aragón* son estaciones distintas de OpenAQ (coordenadas diferentes, 4 sensores cada una), por lo que se conservan separadas	Pueden confundirse al leer reportes por zona

1.5. Índices

Las consultas filtran por fecha, sensor, zona y umbrales de contaminación, por lo que se crearon los índices de la tabla 6. Los índices definidos sobre la tabla particionada `medicion` se propagan automáticamente a cada partición (por ejemplo, `medicion_2024_fecha_idx`).

Cuadro 6: Índices creados en PostgreSQL.

Índice	**Tabla**	**Columna**	**Consulta que acelera**
`medicion_pkey`	`medicion`	`(id_medicion, fecha)`	Unicidad de cada medición
`idx_medicion_fecha`	`medicion`	`fecha`	Rangos de fechas y series temporales
`idx_medicion_sensor`	`medicion`	`id_sensor`	Lecturas de un sensor y uniones con `sensor`
`idx_medicion_pm25`	`medicion`	`pm25`	Filtros por umbral de contaminación
`idx_sensor_zona`	`sensor`	`id_zona`	Sensores por zona
`idx_sensor_organismo`	`sensor`	`id_organismo`	Sensores por organismo
`idx_zona_nombre`	`zona`	`nombre_zona`	Búsqueda por nombre (función `sensores_por_zona`)
`idx_resumen_zonas`	`resumen_zonas`	`nombre_zona`	Consultas sobre la vista materializada

Además, las restricciones `UNIQUE` crean índices implícitos sobre los nombres de sensor, ciudad, categoría y organismo, y sobre el correo de los usuarios. El costo de los índices es mayor espacio en disco y escrituras más lentas, lo cual es aceptable porque las mediciones se insertan una vez y se consultan muchas veces.

1.6. Particionamiento

La tabla `medicion` se particionó por rango de `fecha`, con una partición por año (2016 a 2026) y una partición `default` para fechas fuera de rango.

Se eligió este criterio porque las mediciones son series temporales que casi siempre se consultan por periodo; así, una consulta con un rango de fechas solo lee las particiones que lo cubren (*partition pruning*) y los años antiguos pueden archivarse o descartarse sin tocar el resto.

La tabla 7 muestra la distribución real de las filas y el espacio que ocupa cada partición.

Cuadro 7: Filas y tamaño por partición de `medicion`.

Partición	**Filas**	**% del total**	**Tamaño total (con índices)**
`medicion_2016`	387.578	36,5	51 MB
`medicion_2017`	6.401	0,6	1,1 MB
`medicion_2018`	10.051	0,9	1,8 MB
`medicion_2019`	9.482	0,9	1,8 MB
`medicion_2020`	122.869	11,6	16 MB
`medicion_2021`	180.774	17,0	24 MB
`medicion_2022`	2.142	0,2	0,5 MB
`medicion_2023`	86.397	8,1	12 MB
`medicion_2024`	20.018	1,9	3,1 MB
`medicion_2025`	28.215	2,7	4,2 MB
`medicion_2026`	208.931	19,7	36 MB
`medicion_default`	0	0,0	32 kB
Total	**1.062.858**	**100,0**	

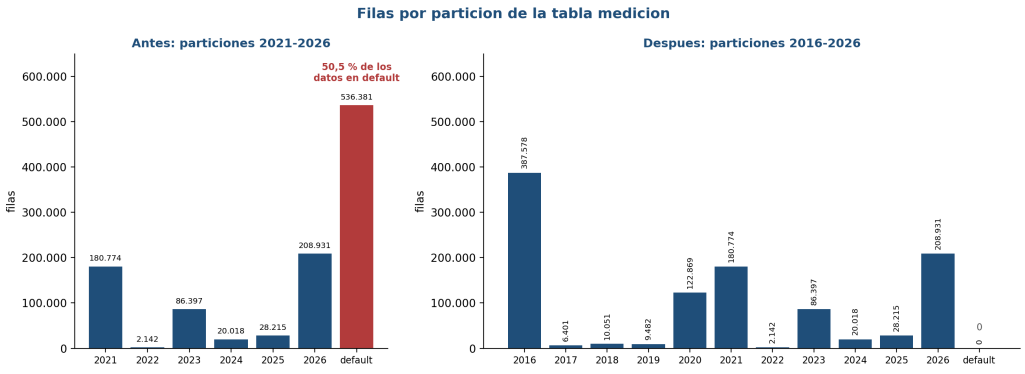


Figura 4: Filas por partición antes y después de crear las particiones 2016–2020.

Problemas encontrados durante la migración. Convertir una tabla existente en particionada no es un solo comando, y el proceso dejó cinco problemas que se detectaron y corrigieron (tabla 8).

Cuadro 8: Problemas detectados al particionar `medicion` y su corrección.

Problema	Causa	Corrección
El 50,5 % de las filas (536.381) estaba en la partición `default`	Solo se habían creado las particiones 2021–2026 y el aire empieza en 2016	Se crearon las particiones 2016–2020 y se movieron las filas con una tabla temporal dentro de una transacción. `default` quedó en 0
La llave foránea hacia `sensor` se perdió	La tabla particionada se creó sin `REFERENCES`	Se agregó `fk_medicion_sensor` con `ALTER TABLE`
`resumen_zonas` seguía apuntando a la tabla original	Al renombrar la tabla antigua a `medicion_backup`, la vista conservó su referencia (los objetos se enlazan por identificador, no por nombre). El `DROP TABLE` del respaldo falló por esa dependencia	Se borró la vista, se eliminó el respaldo y se recreó la vista sobre la tabla particionada
La secuencia de `id_medicion` empezaba en 1	Los identificadores se copiaron explícitos, pero la secuencia de la tabla nueva no avanzó	Se ajustó con `setval` al máximo actual (1.062.858)
La llave primaria debe incluir la columna de partición	Restricción de PostgreSQL	Llave primaria `(id_medicion, fecha)`

1.7. Vista materializada y función almacenada

La vista materializada `resumen_zonas` guarda el resultado de una agregación costosa (promedios y conteos sobre más de un millón de filas) para

consultarla al instante. Su costo es que no se actualiza sola: hay que ejecutar `REFRESH MATERIALIZED VIEW` cuando cambian las mediciones.

Listing 1: Vista materializada `resumen_zonas`.

```
CREATE MATERIALIZED VIEW resumen_zonas AS
SELECT z.nombre_zona,
       AVG(m.pm25)             AS promedio_pm25,
       AVG(m.carros)           AS promedio_carros,
       COUNT(*)                AS total_mediciones,
       MAX(m.congestion_score) AS max_congestion
FROM medicion m
JOIN sensor s ON m.id_sensor = s.id_sensor
JOIN zona   z ON s.id_zona   = z.id_zona
GROUP BY z.nombre_zona;
```

La función `sensores_por_zona` encapsula una consulta que recibe el nombre de una zona y devuelve sus sensores.

Listing 2: Función almacenada `sensores_por_zona`.

```
CREATE OR REPLACE FUNCTION sensores_por_zona(p_zona VARCHAR)
RETURNS TABLE(nombre_sensor VARCHAR, tipo VARCHAR) AS $$
BEGIN
    RETURN QUERY
    SELECT s.nombre::VARCHAR, s.tipo_sensor::VARCHAR
    FROM sensor s
    JOIN zona z ON s.id_zona = z.id_zona
    WHERE z.nombre_zona = p_zona;
END;
$$ LANGUAGE plpgsql;
```

1.8. Consultas obligatorias

Las diez consultas obligatorias están en `04_consultas_obligatorias.sql`. La tabla 9 resume cada una y su resultado.

Cuadro 9: Consultas obligatorias de PostgreSQL y resultados.

N.°	Consulta	Resultado
1	Sensores por tipo	`calidad_aire` 184, `trafico` 100 y `test` 1
2	Sensores por ubicación (`LEFT JOIN`)	38 zonas; la que más sensores tiene es *Suburban North* (26), seguida de *Financial District* (25) y *Downtown Core* (22)
3	Sensores activos e inactivos	284 activos y 1 inactivo
4	Sensores por organismo responsable	MINAE 185, MOPT 98, Municipalidad 1 y CNFL 1 (suman 285)
5	Ubicaciones con más sensores	*Suburban North* 26, *Financial District* 25, *Downtown Core* 22, *Tech Park* 11; las demás del ranking tienen 10
6	Sensores instalados por año	2016: 81; 2017: 2; 2019: 1; 2020: 76; 2021: 1; 2022: 9; 2023: 7; 2024: 6; 2025: 2; 2026: 100
7	Sensores sin actividad registrada	Únicamente `S-TEST-INACTIVO`
8	Ranking de zonas monitoreadas (`RANK`)	*Suburban North* (53.040 mediciones) es la primera; *Tlalnepantla*, *Camarones* y *Pedregal* empatan en el segundo lugar con 50.000
9	Consulta usando la vista materializada	PM2.5 promedio más alto: *Mexico City* (32,47), *UNAM* (30,01) y *Rincón del Bosque* (28,35)
10	Consulta usando la función almacenada	`sensores_por_zona('Financial District')` devuelve 25 sensores y `sensores_por_zona('Santa Fe')`, 10

La consulta 6 usa `fecha_instalacion`, que se derivó de la primera medición de cada sensor (tabla 5); por eso 2026 concentra los 100 sensores de tráfico. En la consulta 8, `RANK` deja huecos tras un empate (de ahí los lugares 1, 2, 2, 2 y 5).

1.9. Consultas avanzadas

1.9.1. Expresión de tabla común (CTE)

La consulta calcula primero el PM2.5 promedio de cada zona y luego conserva solo las zonas por encima del promedio de todas ellas. Sin CTE habría que repetir la agregación dentro de la condición.

Listing 3: CTE: zonas con PM2.5 superior al promedio general.

```
WITH promedio_por_zona AS (
    SELECT z.nombre_zona, AVG(m.pm25) AS promedio_pm25
    FROM zona z
    JOIN sensor s ON s.id_zona = z.id_zona
    JOIN medicion m ON m.id_sensor = s.id_sensor
    WHERE m.pm25 IS NOT NULL
    GROUP BY z.nombre_zona
)
SELECT nombre_zona, ROUND(promedio_pm25::numeric, 2) AS promedio_pm25
FROM promedio_por_zona
WHERE promedio_pm25 > (SELECT AVG(promedio_pm25) FROM promedio_por_zona)
ORDER BY promedio_pm25 DESC;
```

Devuelve 11 zonas, encabezadas por *Mexico City* (32,47), *UNAM* (30,01) y *Rincón del Bosque* (28,35).

1.9.2. Funciones de ventana

Se usaron tres funciones de ventana, cada una con un propósito distinto.

Promedio móvil de 7 días. Suaviza la serie diaria de PM2.5 de cada zona. La ventana `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW` promedia el día actual y los seis anteriores.

Listing 4: Promedio móvil de 7 días del PM2.5 por zona (2021).

```
WITH diario AS (
    SELECT z.nombre_zona, m.fecha::date AS dia, AVG(m.pm25) AS pm25_dia
    FROM medicion m
    JOIN sensor s ON s.id_sensor = m.id_sensor
    JOIN zona z   ON z.id_zona = s.id_zona
    WHERE m.pm25 IS NOT NULL
      AND m.fecha >= '2021-01-01' AND m.fecha < '2022-01-01'
    GROUP BY z.nombre_zona, m.fecha::date
)
```

```
SELECT nombre_zona, dia, ROUND(pm25_dia::numeric, 2) AS pm25_dia,
       ROUND(AVG(pm25_dia) OVER (PARTITION BY nombre_zona ORDER BY dia
             ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)::numeric, 2) AS
    promedio_movil_7d
FROM diario
ORDER BY nombre_zona, dia;
```

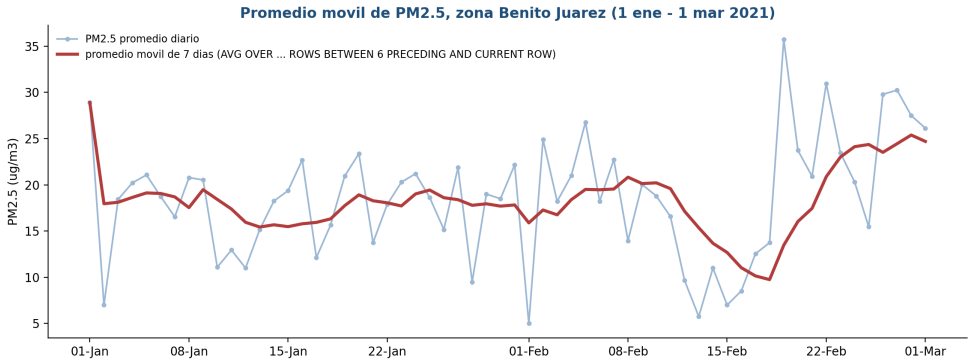


Figura 5: Promedio diario y promedio móvil de 7 días del PM2.5 en la zona Benito Juárez. Origen de los datos: PostgreSQL.

Variación mensual con `LAG`. Compara el PM2.5 de cada mes con el del mes anterior. Como la cobertura del aire tiene huecos, la consulta solo calcula el cambio si el mes anterior existe realmente; de lo contrario compararía, por ejemplo, septiembre contra abril y lo presentaría como variación mensual.

Listing 5: Variación mensual del PM2.5 con LAG.

```
WITH mensual AS (
    SELECT z.nombre_zona, date_trunc('month', m.fecha)::date AS mes,
           AVG(m.pm25) AS pm25_mes
    FROM medicion m
    JOIN sensor s ON s.id_sensor = m.id_sensor
    JOIN zona z   ON z.id_zona = s.id_zona
    WHERE m.pm25 IS NOT NULL
      AND m.fecha >= '2021-01-01' AND m.fecha < '2022-01-01'
    GROUP BY z.nombre_zona, date_trunc('month', m.fecha)
)
SELECT nombre_zona, mes, ROUND(pm25_mes::numeric, 2) AS pm25_mes,
       CASE WHEN LAG(mes) OVER w = (mes - INTERVAL '1 month')::date
            THEN ROUND((pm25_mes - LAG(pm25_mes) OVER w)::numeric, 2)
       END AS cambio_vs_mes_anterior
```

```
FROM mensual
WINDOW w AS (PARTITION BY nombre_zona ORDER BY mes)
ORDER BY nombre_zona, mes;
```

Cuadro 10: Resultado de `LAG` para la zona Benito Juárez (2021).

Mes	PM2.5 promedio	Cambio vs. mes anterior
Enero	18,33	–
Febrero	19,61	+1,28
Marzo	22,96	+3,35
Abril	18,77	−4,19
Septiembre	12,77	– (no hay datos de agosto)

Posición dentro de cada distrito con `RANK`. `RANK() OVER (PARTITION BY distrito ORDER BY pm25_prom DESC)` numera las zonas por contaminación dentro de su distrito. Solo aparece `CDMX` (23 zonas con PM2.5), porque las zonas de `SmartCity District` no tienen datos de aire. *Mexico City* ocupa el primer lugar (32,47) y *Miguel Hidalgo* el último (11,00).

1.9.3. Consultas sobre fechas

Las consultas por rango de fechas aprovechan el particionamiento: la consulta siguiente solo lee la partición `medicion_2024`.

Listing 6: Mediciones por mes durante 2024.

```
SELECT date_trunc('month', fecha)::date AS mes, COUNT(*) AS mediciones
FROM medicion
WHERE fecha >= '2024-01-01' AND fecha < '2025-01-01'
GROUP BY 1
ORDER BY 1;
```

El resultado muestra la cobertura irregular del aire: entre 645 y 2.974 mediciones por mes, salvo septiembre, con apenas 54.

1.9.4. Preguntas de ejemplo del enunciado

Cuadro 11: Preguntas de ejemplo resueltas con PostgreSQL.

Pregunta	**Resultado**
¿Qué zona registra mayor cantidad de vehículos?	*Financial District*: 43.848.854 vehículos; luego *Downtown Core* (37.612.862) y *Suburban North* (33.024.532)
¿Cuál es la temperatura promedio por zona?	Cercana a 15,0 °C en las seis zonas de tráfico (de 14,97 en *Residential West* a 15,02 en *Industrial East*)
¿Cuál es la hora de mayor congestión?	12:00, con congestión promedio de 99,80 y 1.761,1 vehículos; seguida de las 13:00 (99,48) y las 11:00 (99,47)
¿Cuál zona presenta mayor contaminación?	*Mexico City*: PM2.5 promedio de 32,47 sobre 5.000 lecturas; luego *UNAM* (30,01) y *Rincón del Bosque* (28,35)
¿Qué sensores dejaron de enviar información?	Se compara la última medición de cada sensor con la más reciente de toda la red (21 jun 2026) y se listan los que llevan más de 30 días sin datos. Los más antiguos son estaciones de la Ciudad de México cuyo último dato es de noviembre de 2016 (unos 3.500 días). Los sensores de tráfico, que terminan el 26 de marzo de 2026 (87 días antes), también cumplen el criterio. `S-TEST-INACTIVO` aparece sin ninguna medición

La consulta de la zona más contaminada exige al menos 1.000 lecturas por zona (`HAVING`), para no clasificar una zona por un puñado de datos. La temperatura casi idéntica en todas las zonas sugiere que el conjunto de tráfico es, al menos en parte, sintético.

1.10. Resultados

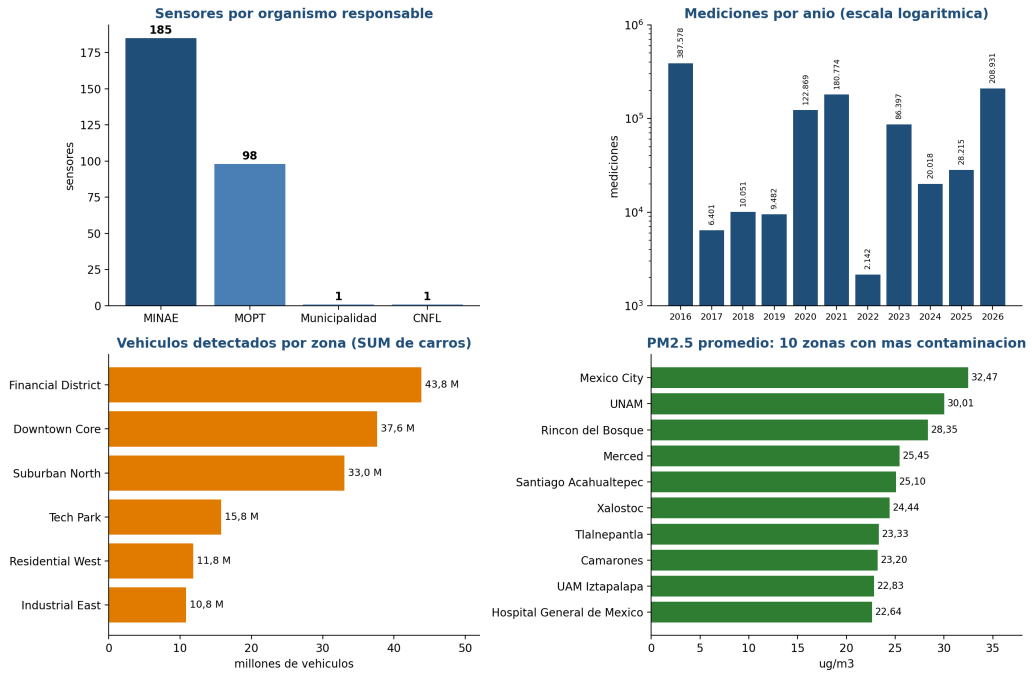


Figura 6: Resultados de consultas en PostgreSQL: sensores por organismo, mediciones por año, vehículos por zona y PM2.5 por zona. Origen de los datos: PostgreSQL.

- **Sensores por organismo.** MINAE y MOPT concentran 283 de los 285 sensores, porque la asignación se hizo por categoría (ambiente y movilidad); el proyecto no tiene sensores de energía, por lo que CNFL no recibe ninguno de forma natural.
- **Mediciones por año.** El histórico es muy desigual: 2016 aporta el 36,5 % de las mediciones y 2022 apenas el 0,2 %. Al comparar años conviene usar promedios y no totales.
- **Vehículos por zona.** Tres zonas (*Financial District*, *Downtown Core* y *Suburban North*) suman 114,5 de los 152,9 millones de vehículos, cerca del 75 %. Esa diferencia en parte refleja que tienen más sensores (25, 22 y 26 frente a 8 y 9 de *Industrial East* y *Residential West*).

- **PM2.5.** Ninguna de las diez zonas más contaminadas supera 33 μg/m^3 de promedio histórico.

1.11. Evaluación del rendimiento: índices y particionamiento

Se ejecutó `EXPLAIN (ANALYZE, BUFFERS)` sobre seis consultas (`06_rendimiento.sql`). La tabla 12 resume los planes.

Cuadro 12: Resultados de `EXPLAIN ANALYZE` (tiempos medidos en la base real del proyecto).

N.°	Consulta	Plan	Tiempo
1	`pm25 >50`, sin índice	`Parallel Seq Scan` en las 12 particiones	14.309 ms
2	`pm25 >50`, con índice	`Index Only Scan` sobre el índice de `pm25` de cada partición	4.489 ms
3	Rango de enero de 2024	*Partition pruning*: solo `medicion_2024`, con `Index Only Scan` (641 filas)	4.359 ms
4	`pm25 >50` sin filtro de fecha	Recorre todas las particiones con índice (`Index Only Scan`)	862 ms
5	Un sensor en un mes de 2021	*Pruning* a `medicion_2021` e `Index Scan` sobre el índice de sensor	4.8 ms
6	`JOIN` de 3 tablas con `AVG` por zona	`Hash Join` y agregación paralela sobre las particiones	34.776 ms

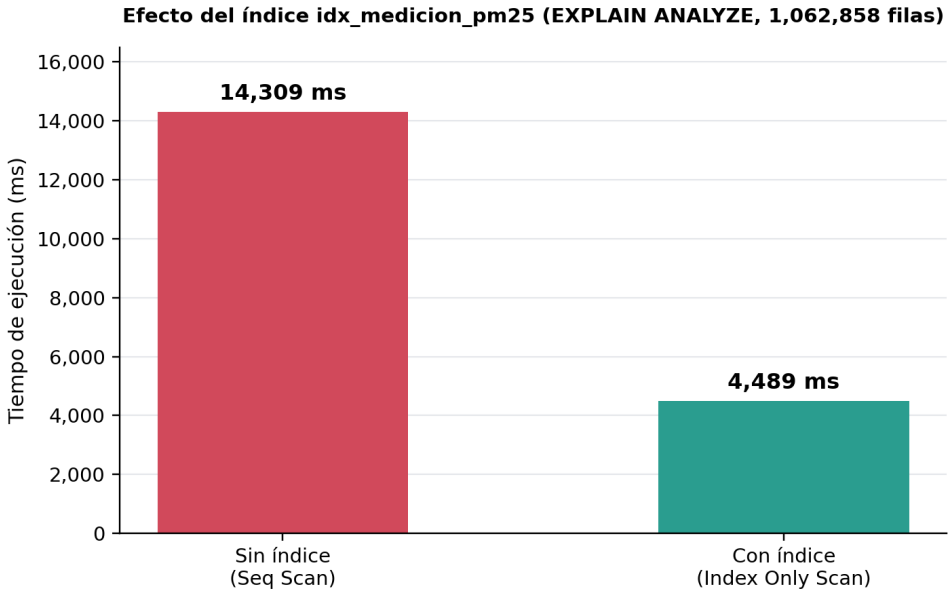


Figura 7: Efecto del índice sobre `pm25`. El tiempo baja de 14.3 segundos a 4.4 segundos con el índice. Origen de los datos: PostgreSQL (`EXPLAIN ANALYZE`).

Interpretación.

- **El índice reduce el tiempo unas 3 veces** (de 14.309 ms a 4.489 ms) y los bloques leídos drásticamente. El filtro `pm25 >50` es muy selectivo (devuelve 5.424 de las 1.062.858 filas, el 0,5 %), por lo que el índice vale la pena.
- **El *partition pruning* funciona.** En la consulta 3 el plan menciona una sola partición de las doce; las demás ni se abren.
- **El planificador decide por partición.** En la consulta 6, que lee casi todas las filas con PM2.5, usa `Hash Join` y agregación paralela sobre las particiones, siendo la consulta más pesada (34.7 segundos) porque involucra un cruce de 1 millón de registros.
- **Mejora posible (no implementada).** Un índice compuesto `(id_sensor, fecha)` evitaría descartar filas por el filtro de fecha en la consulta 5.

1.12. Evaluación: ventajas, limitaciones y escalabilidad

Cuadro 13: Valoración de PostgreSQL según la necesidad, con evidencia del proyecto.

Necesidad	Evidencia	Valoración
Datos estructurados	Siete tablas con llaves primarias, foráneas y restricciones	Excelente
Relaciones complejas	Uniones de cuatro tablas (sensor, zona, categoría y organismo) y cinco relaciones 1:N	Excelente
Integridad relacional	Llaves foráneas y restricciones `UNIQUE` y `NOT NULL`; imposible registrar una medición de un sensor inexistente	Excelente
Analítica con SQL	CTE, funciones de ventana, subconsultas y vista materializada	Excelente
Grandes volúmenes	1.062.858 filas con consultas en milisegundos gracias a índices y particiones	Buena, con límites
Datos semiestructurados	Tabla ancha con `NULL`; un campo nuevo exige `ALTER TABLE`	Limitada
Escalado horizontal	No implementado; el escalado es vertical y por particiones en un solo servidor	Limitado

Limitaciones.

- La tabla `medicion` es ancha: aire y tráfico comparten columnas y cada fila deja en `NULL` las que no le corresponden. Es el contraste con MongoDB, donde cada tipo de sensor guarda solo sus campos (sección 2).
- Las dos fuentes no comparten zonas y el tráfico cubre solo 85 días, lo que impide un análisis conjunto por zona.
- El conjunto de tráfico es en parte sintético (temperatura casi idéntica en todas las zonas) y el aire tiene cobertura irregular.
- La vista materializada no se actualiza sola.

Si el volumen aumentara 100 veces. Serían unos 106 millones de filas. Las propuestas de diseño son:

- mantener el particionamiento por fecha y particionar también por mes los años con más datos;
- archivar las particiones antiguas (`DETACH PARTITION`) y trasladarlas a HDFS, que es más apropiado para almacenamiento masivo y barato;
- usar un índice `BRIN` sobre `fecha`, mucho más pequeño que un `B-tree` para datos que se insertan en orden cronológico;
- cargar con `COPY` en lugar de `INSERT` y programar el refresco de la vista materializada;
- para análisis sobre todo el histórico, delegar en Spark, y si hiciera falta repartir los datos entre servidores, una extensión de fragmentación como Citus.

Son propuestas de diseño; no se implementaron en este proyecto.

1.13. Conclusiones de la parte relacional

- PostgreSQL es la tecnología adecuada para los datos maestros y las relaciones de SmartCity: sensores, zonas, categorías y organismos requieren integridad referencial y consultas con `JOIN`.
- El modelo se llevó hasta 3FN con una corrección real (la relación entre organismo y sensor) y una desviación explicada (la llave compuesta de `medicion`, exigida por el particionamiento).
- El particionamiento por fecha y los índices mantienen las consultas en milisegundos con más de un millón de filas, y su ventaja crece con el volumen.
- Los problemas de la migración (particiones faltantes, llave foránea perdida, vista apuntando al respaldo, secuencia desfasada) muestran que particionar una tabla existente exige verificar los objetos que dependen de ella.
- El diseño relacional pierde flexibilidad frente a datos que cambian de forma, y no escala horizontalmente por sí solo. Esas necesidades se cubren con MongoDB, HDFS y Spark.

2. Base de datos documental: MongoDB

2.1. Rol de MongoDB en la arquitectura

MongoDB almacena las lecturas de los sensores, los eventos detectados, las alertas automáticas y los incidentes urbanos de SmartCity. Las entidades maestras y de referencia (sensores, ubicaciones, categorías, organismos responsables) pertenecen a PostgreSQL; MongoDB guarda la información que los sensores generan de forma continua y que cambia de forma de un sensor a otro.

La elección de un modelo documental se justifica por cinco características de estos datos:

- **Estructura heterogénea.** Los sensores de tráfico miden vehículos, congestión, emisiones y temperatura; los de calidad del aire miden PM2.5, PM10, NO_2, O_3 y CO. En un modelo relacional esto exige una tabla muy ancha con muchas columnas `NULL` o una tabla por tipo de sensor. En MongoDB ambos tipos conviven en la misma colección, cada uno con su subdocumento `medicion`.
- **Datos anidados que se leen juntos.** Cada lectura contiene el sensor (identificador, zona, ubicación), las mediciones y los eventos asociados. Al estar en un solo documento, la consulta más frecuente ("lecturas de un sensor") no necesita `JOIN`.
- **Esquema flexible.** Los eventos tienen un subdocumento `detalles` cuya estructura depende del tipo de evento. Se pueden agregar campos nuevos sin modificar el esquema (sección 2.4).
- **Escritura continua y consultas por sensor y tiempo.** Los datos se insertan sin modificarse después, y se consultan casi siempre por sensor, zona o rango de fechas, que son patrones que un índice compuesto resuelve bien.
- **Analítica con el Aggregation Framework.** Los promedios por zona, los máximos y mínimos y las series temporales se expresan como pipelines de agregación directamente sobre los documentos.

MongoDB no es la mejor opción para relaciones complejas ni para garantizar integridad referencial; por eso esas tareas quedan en PostgreSQL (sección 2.8).

2.2. Fuentes de datos y volumen

Se integraron dos fuentes distintas, como pide el enunciado. La tabla 14 resume su contenido tal como quedó almacenado en MongoDB.

Cuadro 14: Fuentes de datos almacenadas en la colección `lecturas`.

Fuente	Archivo	Contenido	Periodo	Documentos
Tráfico vehicular	`smart_city_traffic_mobility.csv`	Vehículos, congestión, temperatura, AQI, emisiones y accidentes de 100 intersecciones (`INT_xxx`) en 6 zonas; una lectura por hora	1 ene – 26 mar 2026 (85 días)	204.000
Calidad del aire (CDMX)	`datos_aire_cdmx.csv`	PM2.5, PM10, NO_2, O_3 y CO de sensores de la Ciudad de México (`CDMX_xxx`)	oct 2020 – 2025 (sin datos en 2022)	296.000

Fuentes de los datos. Las fuentes son las mismas utilizadas en PostgreSQL: el dataset de tráfico de Kaggle (Mobeen Fatima, CC BY 4.0) y las lecturas de la API de OpenAQ v3. En MongoDB se almacena un subconjunto de 500.000 documentos derivados del proceso de limpieza (`limpiar_datos.py`) para demostrar la flexibilidad del modelo documental frente a los datos anidados.

https://explore.openaq.org/locations/1134download-card

El volumen total almacenado en MongoDB es de 655.535 documentos: 500.000 lecturas, 79.428 alertas, 72.078 eventos y 4.029 incidentes. Esto cumple el escenario mínimo del enunciado (500.000 documentos en MongoDB) y no alcanza el escenario recomendado de más de 5 millones de registros totales.

Las dos fuentes no coinciden en el tiempo. Los datos de aire cubren de 2020 a 2025 y los de tráfico solo de enero a marzo de 2026, por lo que no existe ningún día en común (figura 8). Tampoco comparten zonas: el tráfico usa zonas ficticias (*Financial District*, *Downtown Core*, etc.) y el aire usa zonas de la Ciudad de México (*Merced*, *Pedregal*, etc.). Por esta razón las fuentes no se cruzan por fecha ni por zona, sino por patrón horario (sección 2.7).

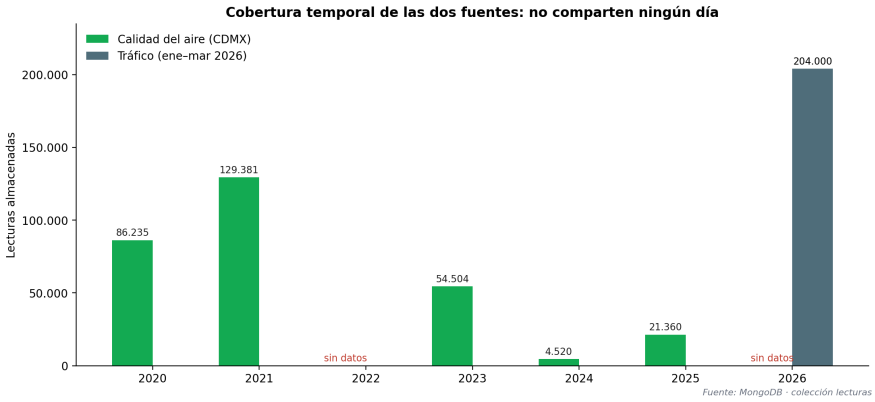


Figura 8: Cobertura temporal de las dos fuentes. Origen de los datos: MongoDB.

2.3. Flujo de ingestión y generación de datos

La figura 9 muestra el flujo completo hasta MongoDB. Los pasos son:

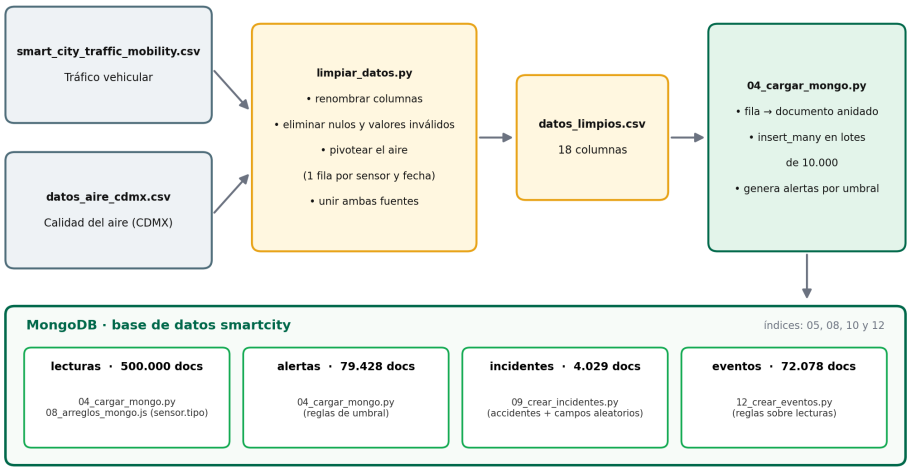
1. **Limpieza** (`limpiar_datos.py`). Se renombran las columnas al esquema común, se eliminan filas con fecha, zona o sensor nulos y vehículos negativos, y se *pivotea* el dataset de aire: el original trae una fila por cada contaminante, y se transforma a una fila por sensor y fecha con una columna por contaminante. Ambas fuentes se unen en `datos_limpios.csv`, con 18 columnas.

2. **Carga** (`04_cargar_mongo.py`). Cada fila se convierte en un documento anidado (`sensor`, `medicion`, `eventos`, `timestamp`) y se inserta con `insert_many` en lotes de 10.000 para limitar el uso de memoria. Los campos que no aplican a un tipo de sensor se almacenan como `null`.

3. **Clasificación** (`08_arreglos_mongo.js`). Se agrega el campo `sensor.tipo` con `updateMany`: los identificadores que empiezan por `CDMX_` se marcan como `calidad_aire` y el resto como `trafico`.

4. **Colecciones derivadas.** Las alertas se generan en la carga; los incidentes con `09_crear_incidentes.py`; los eventos con `12_crear_eventos.py`.

5. **Índices** (`05_mongo_indices.js`, `10_indices_incidentes.js` y `12_crear_eventos.py`).



Flujo de ingestión hacia MongoDB

Figura 9: Flujo de ingestión desde los archivos CSV hasta las colecciones de MongoDB.

2.3.1. Datos derivados y sintéticos

El enunciado exige documentar el mecanismo con el que se generaron los datos que no vienen directamente de las fuentes. La tabla 15 lo detalla.

Cuadro 15: Origen de cada colección y campo derivado.

Colección o campo	**Origen**	**Mecanismo**	**Reproducible**
`lecturas`	Datos de las fuentes	Transformación de los CSV limpios a documentos anidados	Sí
`sensor.tipo`	Derivado	Se deduce del prefijo del identificador (`CDMX_` o no)	Sí
`alertas`	Derivado por reglas	Se evalúa cada lectura: PM2.5 $>$ 50 genera `contaminacion_alta` (crítica si $>$ 100); `congestion_score` $>$ 70 genera `congestion_alta` (crítica si $>$ 90)	Sí
`eventos`	Derivado por reglas	Una lectura genera un evento si PM2.5 $>$ 100, congestión $>$ 90 o hay accidente. Cada tipo guarda solo los campos que le aplican en `detalles`	Sí
`incidentes`	Sintético a partir de datos reales	Se crea un incidente por cada lectura con accidente reportado (4.029). El tipo, la gravedad, los vehículos involucrados (1–5) y el tiempo de respuesta (5–60 min) se asignan con el módulo `random` de Python, sin semilla fija	No exacta

Como el tipo y la gravedad de los incidentes se asignan al azar, su distribución es casi uniforme y no representa un fenómeno real; solo la zona, la ubicación y la fecha provienen de los datos originales.

2.4. Modelo documental

La base de datos `smartcity` tiene cuatro colecciones (figura 10).

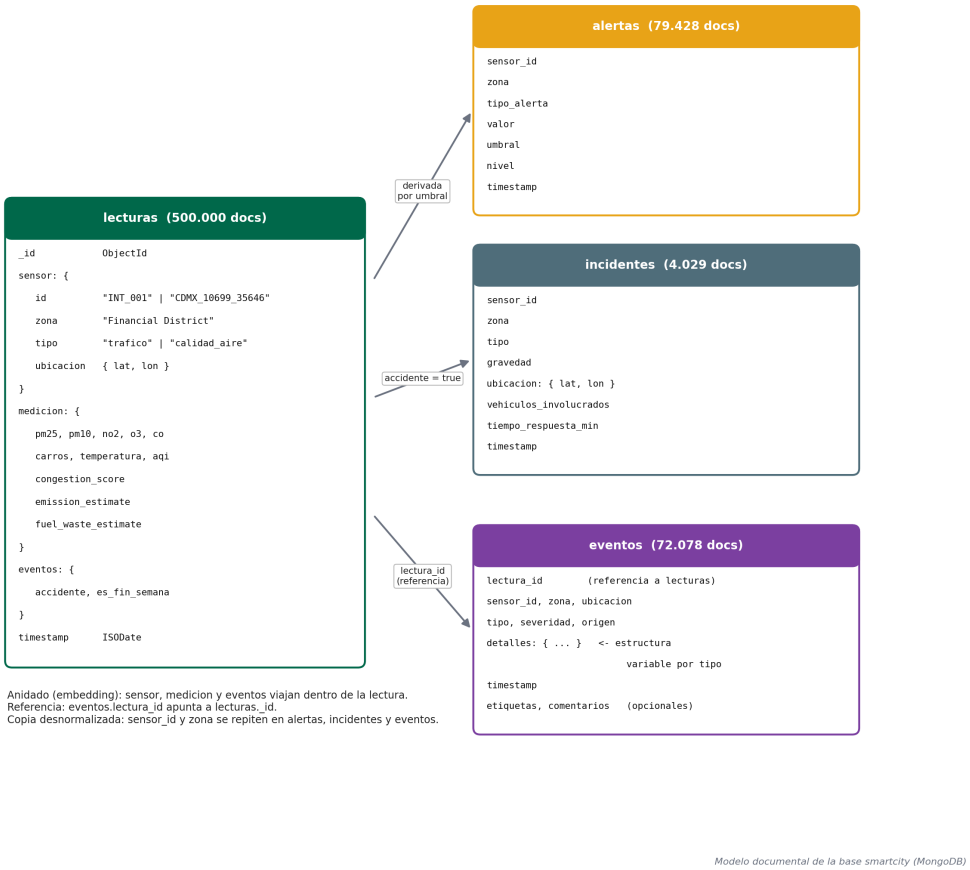


Figura 10: Modelo documental de la base `smartcity`.

Decisiones de diseño.

- **Anidar** `sensor`, `medicion` y `eventos` dentro de `lecturas`: siempre se consultan juntos, cada lectura es pequeña (muy por debajo del límite de 16 MB por documento) y se evita el `JOIN`.
- **Colecciones separadas** para alertas, incidentes y eventos: tienen un ciclo de vida, un volumen y patrones de consulta distintos a los de las lecturas. Una consulta de alertas por zona no necesita recorrer 500.000 lecturas.
- **Referencia** en `eventos`: el campo `lectura_id` apunta a la lectura que originó el evento, para poder recuperarla sin duplicar todas sus

mediciones.

- **Desnormalización controlada**: `sensor_id` y `zona` se copian en alertas, incidentes y eventos para filtrar y agrupar sin consultar la colección de lecturas. El costo es duplicar esos campos.

Ejemplos de documentos. Los valores son ilustrativos y respetan la estructura real de cada colección.

Listing 7: Lectura de un sensor de tráfico (colección `lecturas`).

```
{
  "sensor": { "id": "INT_001", "zona": "Financial District",
              "tipo": "trafico",
              "ubicacion": { "lat": <lat>, "lon": <lon> } },
  "medicion": { "pm25": null, "pm10": null, "no2": null,
                "o3": null, "co": null,
                "carros": 162, "temperatura": 11.1, "aqi": 52,
                "congestion_score": 16.8,
                "emission_estimate": 39.14,
                "fuel_waste_estimate": 8.53 },
  "eventos": { "accidente": false, "es_fin_semana": false },
  "timestamp": ISODate("2026-01-01T00:00:00Z")
}
```

Listing 8: Alerta automática (colección `alertas`).

```
{
  "sensor_id": "INT_096", "zona": "Financial District",
  "tipo_alerta": "congestion_alta", "valor": 100.0,
  "umbral": 70, "nivel": "critico",
  "timestamp": ISODate("2026-02-18T10:00:00Z")
}
```

Información dinámica no estructurada. El subdocumento `detalles` de los eventos cambia según el tipo, y se pueden insertar eventos con campos que no existían antes, incluidos arreglos y subdocumentos, sin modificar el esquema (`13_info_dinamica.js`). El listado siguiente muestra tres eventos con estructuras distintas en la misma colección:

Listing 9: Eventos con estructuras diferentes en la misma colección.

```
{ "tipo": "obra_vial", "zona": "Financial District",
```

```
  "detalles": { "carriles_cerrados": 2, "duracion_estimada_h": 6 } }

{ "tipo": "evento_masivo", "zona": "Financial District",
  "detalles": { "asistentes_estimados": 15000,
                "vias_cerradas": ["Avenida 1", "Calle 5"],
                "organizador": "Municipalidad" },
  "etiquetas": ["concierto", "trafico"] }

{ "tipo": "falla_sensor", "sensor_id": "INT_001",
  "detalles": { "ultimo_reporte": ISODate("..."), "bateria_pct": 12 },
  "comentarios": [ { "autor": "tecnico1",
                     "texto": "Revisar alimentacion" } ] }
```

La consulta con `$exists` sobre `detalles.carriles_cerrados` devolvió únicamente el evento de obra vial, y la agregación con `$objectToArray` mostró que los tres tipos automáticos tienen conjuntos de campos distintos: la congestión guarda `congestion_score`, `carros`, `emission_estimate` y `fuel_waste_estimate`; el accidente guarda `carros`, `congestion_score`, `temperatura` y `es_fin_semana`; y la contaminación guarda `pm25` y su umbral.

2.5. Índices

Las consultas más frecuentes filtran por sensor, zona, tipo y fecha, por lo que se crearon los índices de la tabla 16.

Cuadro 16: Índices creados en MongoDB.

Colección	Índices	Consulta que acelera
`lecturas`	`sensor.id`; `sensor.zona`; `timestamp` (desc.); `(sensor.id, timestamp desc.)`; `sensor.tipo`	Lecturas de un sensor, promedios por zona, rangos de fechas, filtro por tipo de sensor
`alertas`	`sensor_id`; `zona`; `tipo_alerta`; `timestamp`	Alertas por zona y por tipo
`incidentes`	`sensor_id`; `zona`; `tipo`; `gravedad`; `timestamp`	Incidentes por categoría y gravedad
`eventos`	`tipo`; `zona`; `sensor_id`; `severidad`; `timestamp`; `(zona, tipo)`	Eventos críticos por zona y tipo

Para comprobar el uso del índice se ejecutó `explain(.^ecutionStats")` sobre una consulta de rango de fechas (30 días a partir de la primera lectura, 16 de octubre de 2020). El plan ganador fue `FETCH` sobre `IXSCAN`, y examinó 33.867 documentos para devolver los mismos 33.867: MongoDB recorrió solo el rango del índice y no los 500.000 documentos de la colección. El costo de los índices es mayor espacio en disco y escrituras más lentas, lo cual es aceptable porque los datos se consultan mucho más de lo que se modifican.

2.6. Operaciones CRUD

Cuadro 17: Operaciones CRUD demostradas.

Operación	Archivo	Uso
`insertOne`	`06_mongo_crud.js`	Inserta una lectura de prueba
`insertMany`	`04`, `09`, `12` (`.py`); `13_info_dinamica.js`	Carga por lotes de lecturas, alertas, incidentes y eventos
`find`	`06`, `14_consultas_faltantes.js`	Filtros, proyecciones (inclusión y exclusión) y ordenamiento
`findOne`	`06_mongo_crud.js`, `07`	Recupera un documento de ejemplo
`updateOne`	`06`, `13`	Modifica un campo y agrega campos nuevos con `$set` y `$push`
`updateMany`	`08_arreglos_mongo.js`	Asigna `sensor.tipo` a los 500.000 documentos
`deleteOne`	`06_mongo_crud.js`	Elimina la lectura de prueba

2.7. Consultas de agregación y resultados

Las consultas obligatorias se ejecutan con el Aggregation Framework (`07_mongo_aggregations.js`, `11_aggregations_extra.js` y `14_consultas_faltantes.js`). La tabla 18 resume cada una y su resultado.

Cuadro 18: Consultas obligatorias de MongoDB y resultados.

N.°	Consulta	Resultado
1	Lecturas de un sensor específico	Lecturas horarias de `INT_001`, ordenadas por fecha
2	Lecturas en un rango de fechas	33.867 lecturas entre el 16 oct y el 15 nov 2020; las zonas con más lecturas son *Merced* (3.322) y *Pedregal* (3.259)
3	Valores máximos y mínimos	PM2.5: 157 (`CDMX_10699_35646`, *San Agustín*) y 0; vehículos: 3.162 (`INT_096`, 18 feb 2026) y 54; temperatura: 33,1 y −2,8; congestión: 100 y 5,4
4	Eventos críticos (PM2.5 $>$ 100)	17 eventos: *Santiago Acahualtepec* 11, *San Agustín* 3, *Merced* 2, *Camarones* 1
5	Alertas por zona geográfica	Congestión: *Financial District* 20.589, *Downtown Core* 18.830, *Suburban North* 18.731. Contaminación: *Santiago Acahualtepec* 294
6	Promedio de mediciones por zona	PM2.5 promedio más alto: *Merced* (25,56) y *Santiago Acahualtepec* (25,10)
7	Promedio por tipo de sensor	Tráfico: 204.000 lecturas, 749,6 vehículos promedio. Aire: 296.000 lecturas, PM2.5 promedio 21,47
8	Incidentes por categoría	4.029 incidentes: *atropello* 825, *colisión múltiple* 816, *accidente vehicular* 813, *falla mecánica* 790, *derrape* 785
9	Consulta con agregaciones (sensores con más lecturas)	Varios sensores de aire empatan con 5.000 lecturas cada uno
10	Filtros múltiples	Tráfico en días laborables con más de 100 vehículos y congestión mayor a 70, agrupado por zona

Además se resolvieron las preguntas de ejemplo del enunciado: la temperatura promedio por zona es cercana a 15 °C en las seis zonas de tráfico; los sensores con más excesos del límite de congestión son `INT_096` y `INT_062` (866 lecturas cada uno); y la evolución mensual de los vehículos promedio fue 747,2 (enero), 748,4 (febrero) y 753,7 (marzo).

2.7.1. Alertas por zona

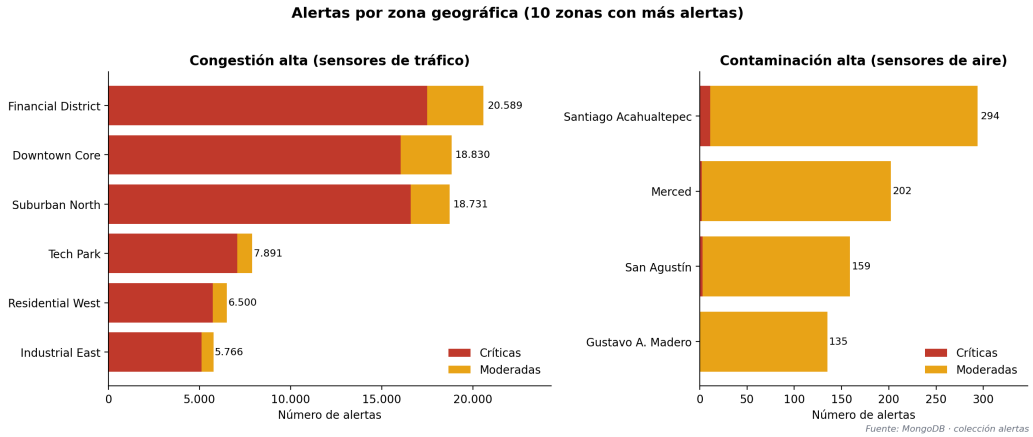


Figura 11: Alertas por zona geográfica. Origen de los datos: MongoDB, colección `alertas`.

Las alertas de congestión (78.307) superan ampliamente a las de contaminación (1.121). Tres zonas (*Financial District*, *Downtown Core* y *Suburban North*) concentran 58.150 de las alertas de congestión, cerca del 74 %, y en todas más del 80 % son críticas.

2.7.2. Patrón horario: tráfico y contaminación

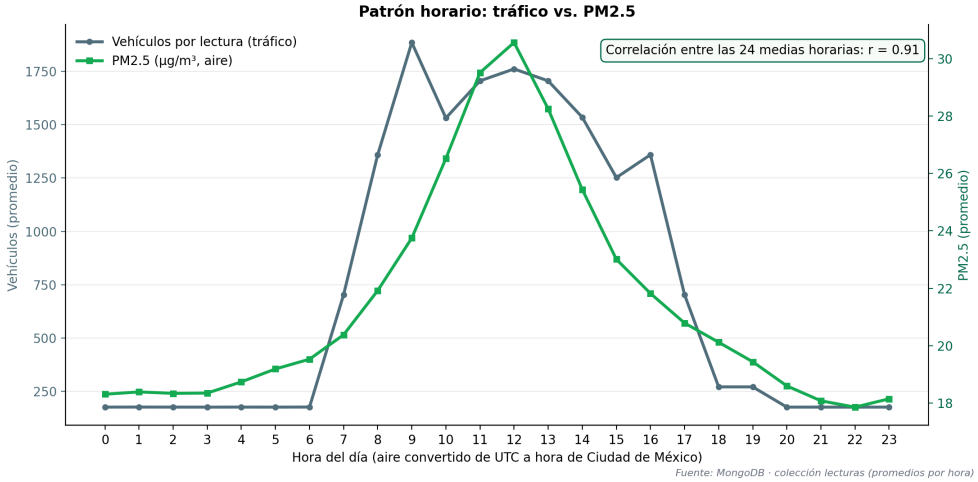


Figura 12: Vehículos y PM2.5 promedio por hora del día. Origen de los datos: MongoDB, colección `lecturas`.

Como las fuentes no coinciden en fechas ni zonas, se compararon por hora del día. El tráfico pasa de unos 177 vehículos en la madrugada a un máximo de 1.885 a las 9 h, y el PM2.5 sube de 18,3 en la madrugada a un máximo de 30,6 a las 12 h. La correlación de Pearson entre las 24 medias horarias es de 0,91. El máximo de PM2.5 llega unas tres horas después del máximo de tráfico, lo que es coherente con la acumulación de contaminantes durante la mañana. Esta cifra describe un patrón diario promedio de dos fuentes de periodos distintos; no demuestra causalidad.

2.7.3. Distribución de lecturas y eventos

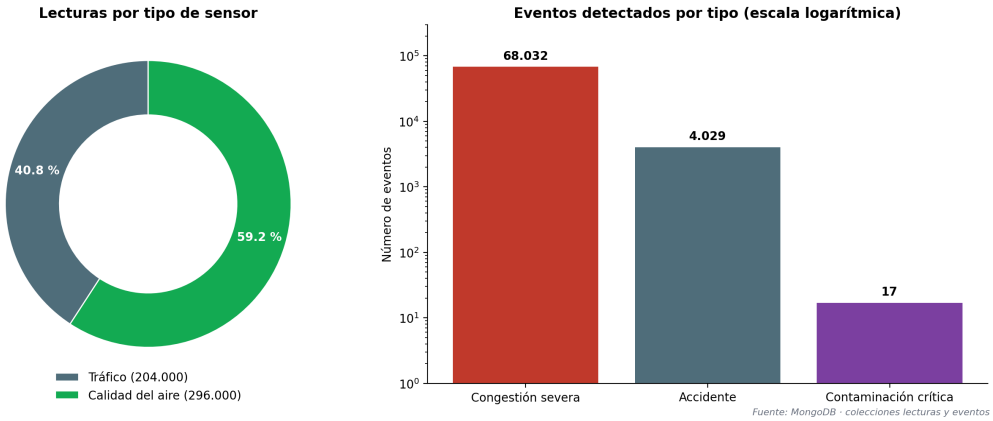


Figura 13: Lecturas por tipo de sensor y eventos por tipo. Origen de los datos: MongoDB, colecciones `lecturas` y `eventos`.

El 40,8 % de las lecturas proviene de sensores de tráfico y el 59,2 % de sensores de aire. De los 72.078 eventos, el 94,4 % son de congestión severa, el 5,6 % accidentes y solo 17 son de contaminación crítica (PM2.5 mayor a 100), lo que indica que el aire de la muestra rara vez supera ese nivel.

2.7.4. Incidentes

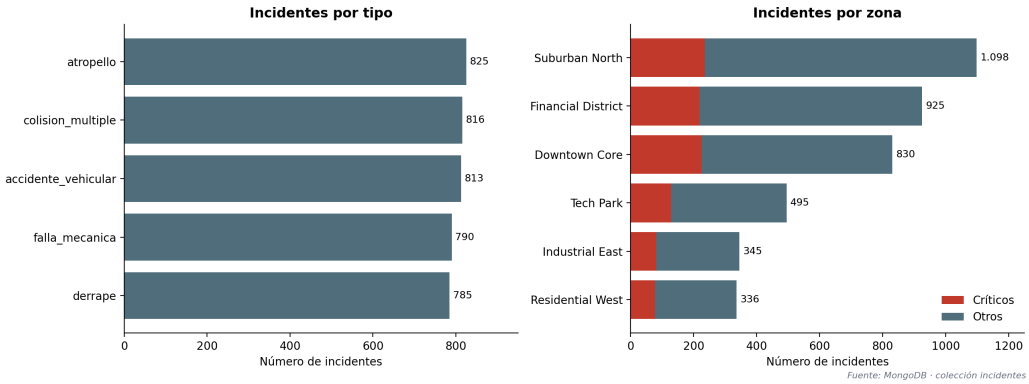


Figura 14: Incidentes por tipo y por zona. Origen de los datos: MongoDB, colección `incidentes`.

La distribución por tipo es casi uniforme (785 a 825) porque el tipo se asignó al azar (tabla 15), por lo que no debe interpretarse como un resultado real. La distribución por zona sí refleja los accidentes de los datos originales: *Suburban North* registra 1.098 incidentes y *Residential West*, 336.

2.8. Evaluación: ventajas, limitaciones y escalabilidad

Cuadro 19: Valoración de MongoDB según la necesidad, con evidencia del proyecto.

Necesidad	**Evidencia**	**Valoración**
Datos semiestructurados	Eventos con `detalles` de estructura distinta en una misma colección, sin cambiar el esquema	Excelente
Consultas por sensor y tiempo	Índice compuesto y rango de fechas con `IXSCAN`	Muy buena
Analítica agregada	Diez consultas con `$group`, `$match`, `$project` sobre 500.000 documentos	Buena
Relaciones entre entidades	Zona y sensor se copian en cada colección; no hay `JOIN` entre fuentes	Limitada
Integridad referencial	`lectura_id` no se valida contra `lecturas`	No aplica

Limitaciones.

- Al unir ambas fuentes en una sola colección, los campos que no aplican a un tipo de sensor se guardan como `null`. Omitirlos reduciría el tamaño de los documentos.
- Las fuentes no comparten fechas ni zonas, de modo que no es posible un análisis conjunto por zona; se cruzan por hora del día.
- Los incidentes son semisintéticos (tabla 15).
- El conjunto de tráfico presenta valores muy homogéneos (por ejemplo, temperatura promedio cercana a 15 °C en todas las zonas), lo que sugiere que es en parte sintético.

Si el volumen aumentara 100 veces. MongoDB escala horizontalmente mediante *sharding*. Una clave de fragmentación candidata es `sensor.id` con hash, que reparte las escrituras entre nodos y mantiene juntas las lecturas de un mismo sensor para su consulta. Para series temporales de este tamaño también convendría usar colecciones de series temporales (*time series*) y trasladar el histórico antiguo a HDFS, que es más apropiado para almacenamiento masivo de bajo costo. Esta es una propuesta de diseño: no se implementó *sharding* en este proyecto.

A. Anexo A: Evidencia de PostgreSQL

A.1. Tablas y conteo de registros

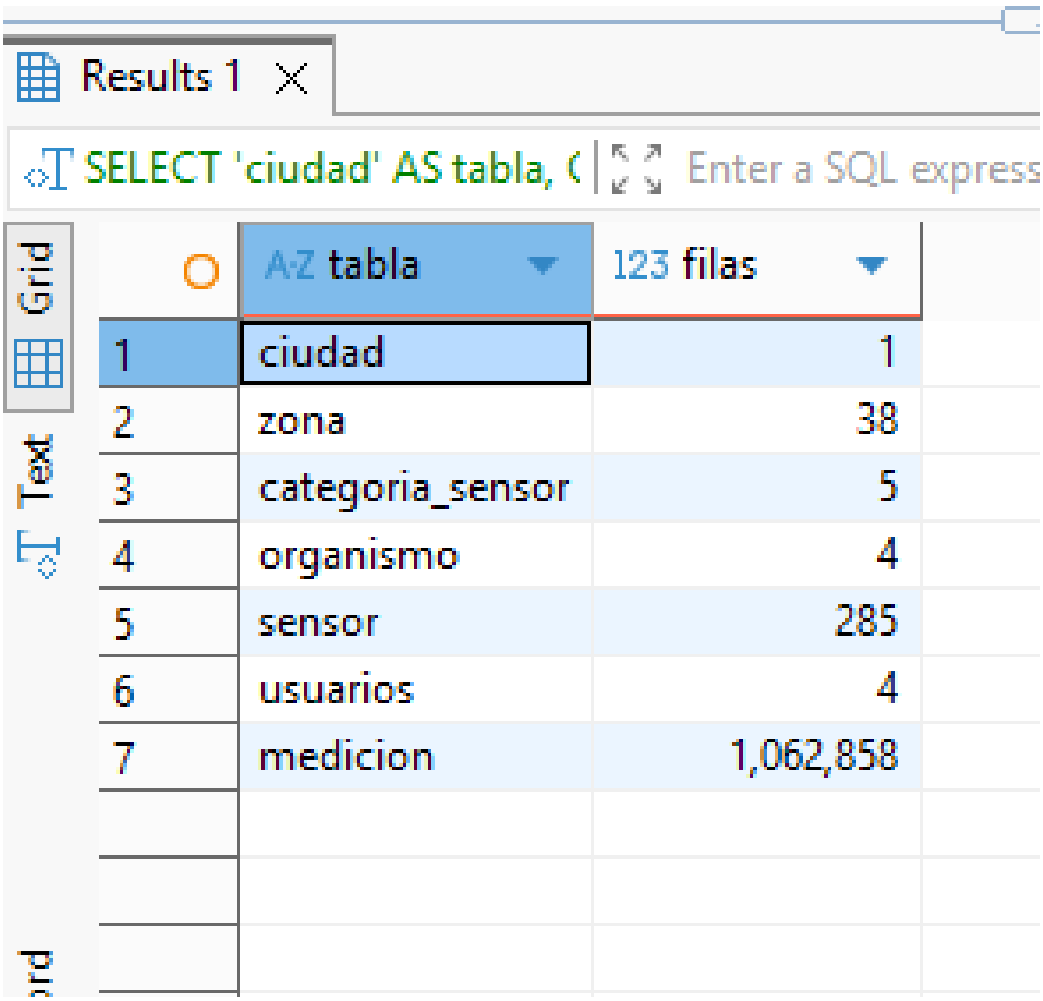


Figura 15: Conteo total de registros por tabla en PostgreSQL (1,062,858 en `medicion`).

A.2. Particionamiento de la tabla `medicion`

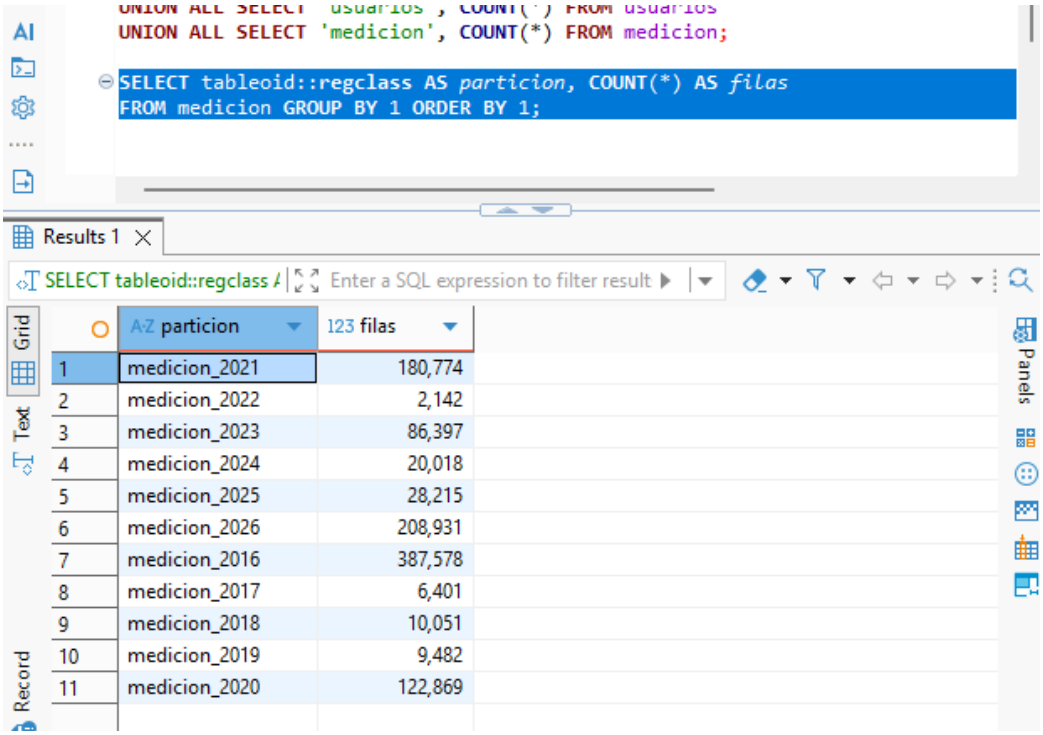


Figura 16: Distribución de filas por año en la tabla particionada `medicion`.

A.3. Diagrama Entidad-Relación

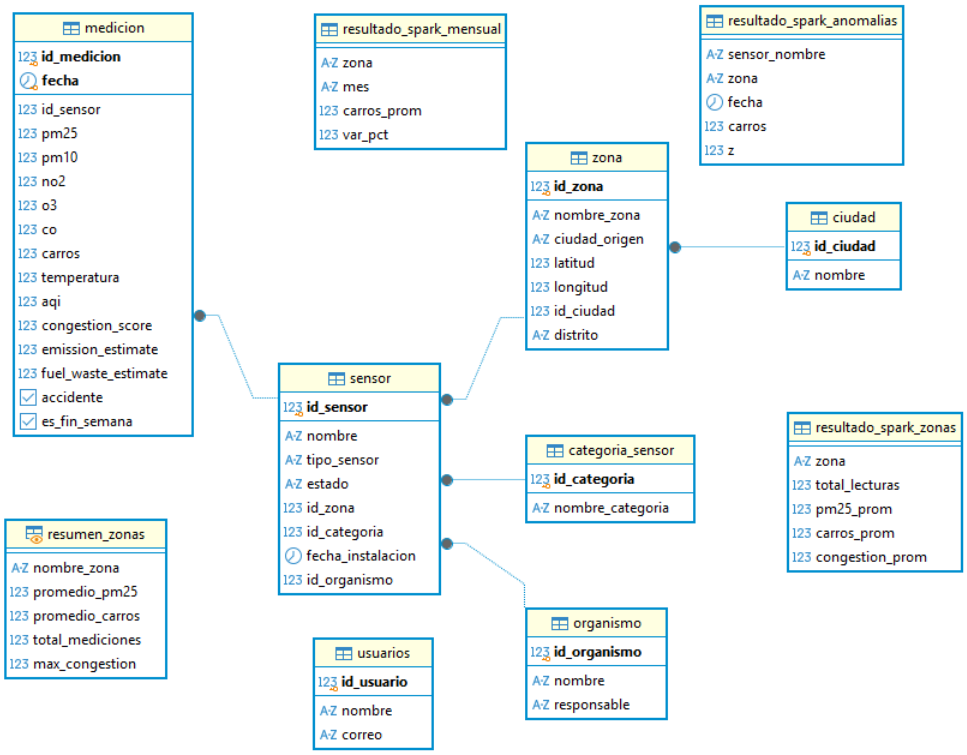


Figura 17: Diagrama Entidad-Relación generado en DBeaver, incluyendo la tabla `resultado_spark_zonas`.

A.4. Llaves foráneas e índices

```
FROM medicion GROUP BY 1 ORDER BY 1;

A4. Llaves e índices (anexo_pg_04_llaves_indices.png)
SELECT conrelid::regclass AS tabla, conname, pg_get_constraintdef(oid) AS definicion
FROM pg_constraint
```

pg_constraint 1 | pg_indexes 1 (2)

SELECT conrelid::regclass AS tabla, conname, pg_get_(Data filter is not supported

	tabla	conname	definicion
1	ciudad	ciudad_pkey	PRIMARY KEY (id_ciudad)
2	zona	zona_id_ciudad_fkey	FOREIGN KEY (id_ciudad) REFERENCES ciudad(id_ciudad)
3	zona	zona_pkey	PRIMARY KEY (id_zona)
4	categoria_sensor	categoria_sensor_pkey	PRIMARY KEY (id_categoria)
5	sensor	fk_sensor_organismo	FOREIGN KEY (id_organismo) REFERENCES organismo(id_organismo)
6	sensor	sensor_id_categoria_fkey	FOREIGN KEY (id_categoria) REFERENCES categoria_sensor(id_categoria)
7	sensor	sensor_id_zona_fkey	FOREIGN KEY (id_zona) REFERENCES zona(id_zona)
8	sensor	sensor_pkey	PRIMARY KEY (id_sensor)
9	organismo	organismo_pkey	PRIMARY KEY (id_organismo)
10	usuarios	usuarios_pkey	PRIMARY KEY (id_usuario)
11	medicion	fk_medicion_sensor	FOREIGN KEY (id_sensor) REFERENCES sensor(id_sensor)
12	medicion	medicion_pkey	PRIMARY KEY (id_medicion, fecha)

Figura 18: Definición de llaves primarias, foráneas e índices en PostgreSQL.

A.5. Pruebas de rendimiento (EXPLAIN ANALYZE)

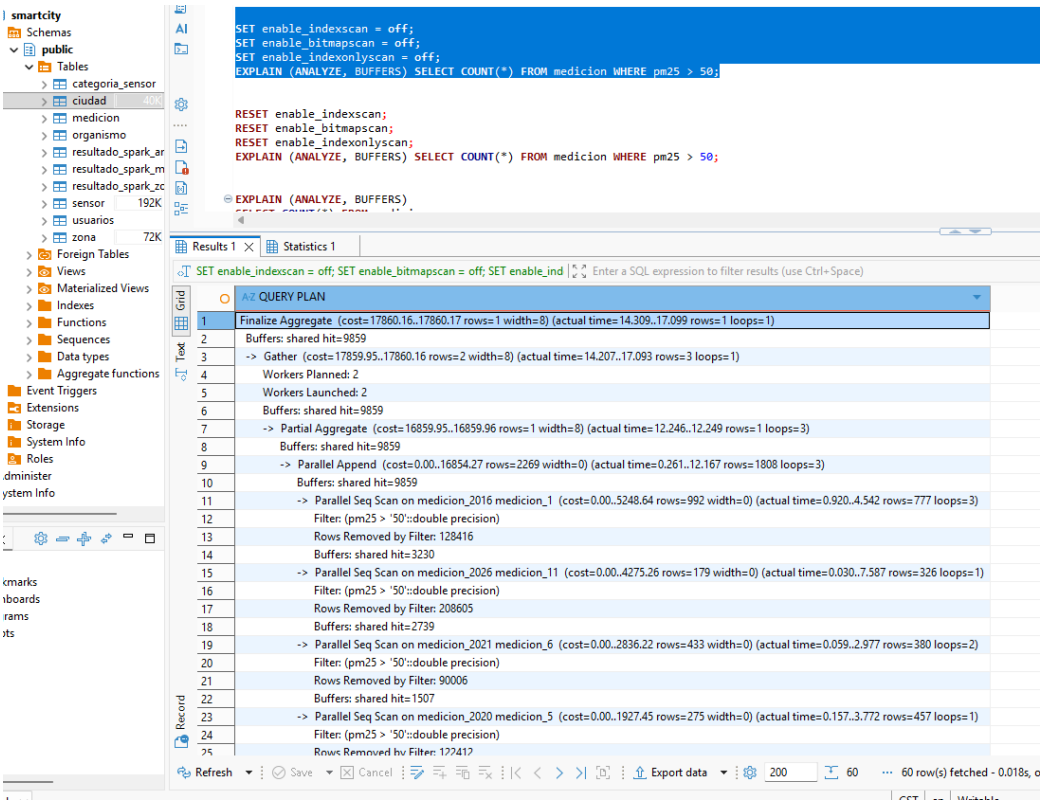


Figura 19: `EXPLAIN ANALYZE` sin índice (Parallel Seq Scan), mostrando un tiempo de 14.3 segundos.

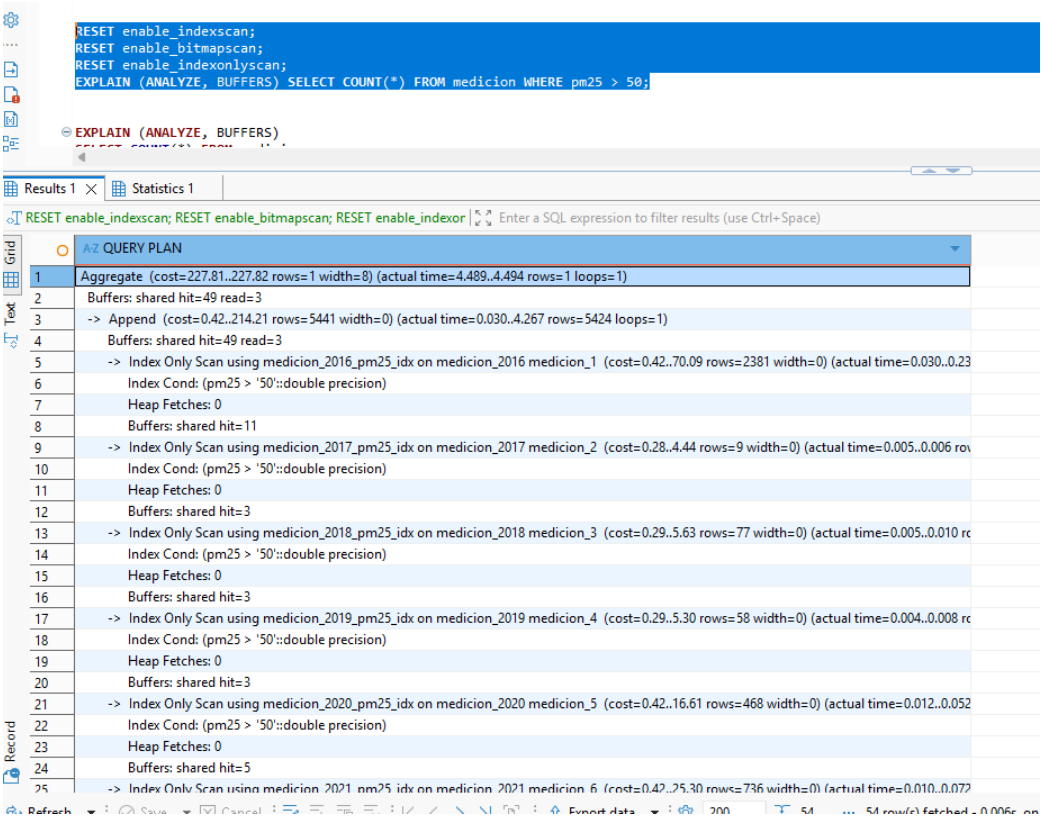


Figura 20: `EXPLAIN ANALYZE` con índice (Index Only Scan), reduciendo el tiempo a 4.4 segundos.

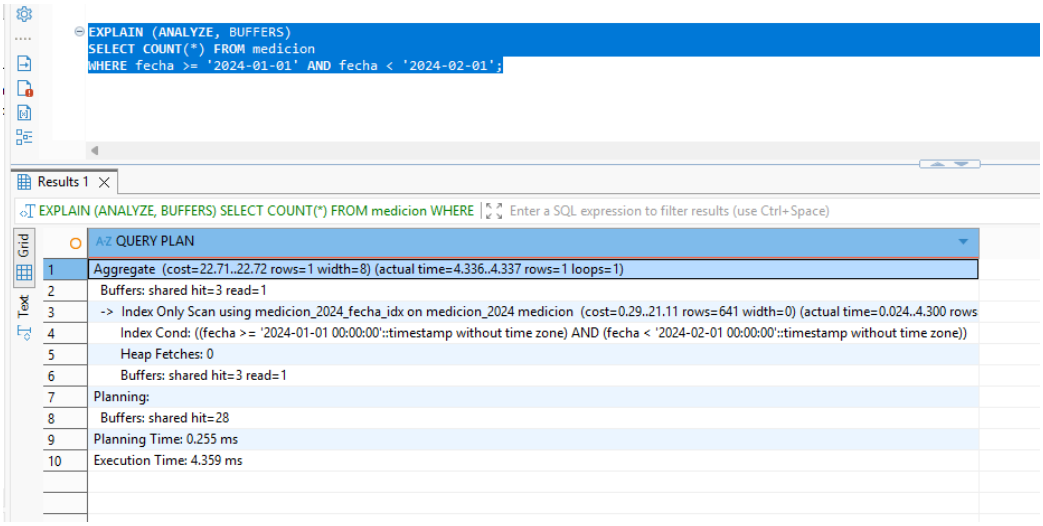


Figura 21: `EXPLAIN ANALYZE` con *Partition Pruning* para enero de 2024, reduciendo el escaneo a una sola partición.

B. Anexo B: Evidencia de MongoDB

B.1. Colecciones y conteo de documentos

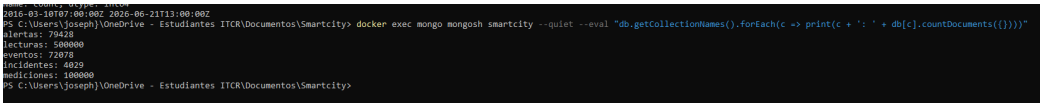


Figura 22: Colecciones en MongoDB y cantidad de documentos (500,000 lecturas, 79,428 alertas).

B.2. Ejecución de una Aggregation (Promedio de PM2.5 por zona)

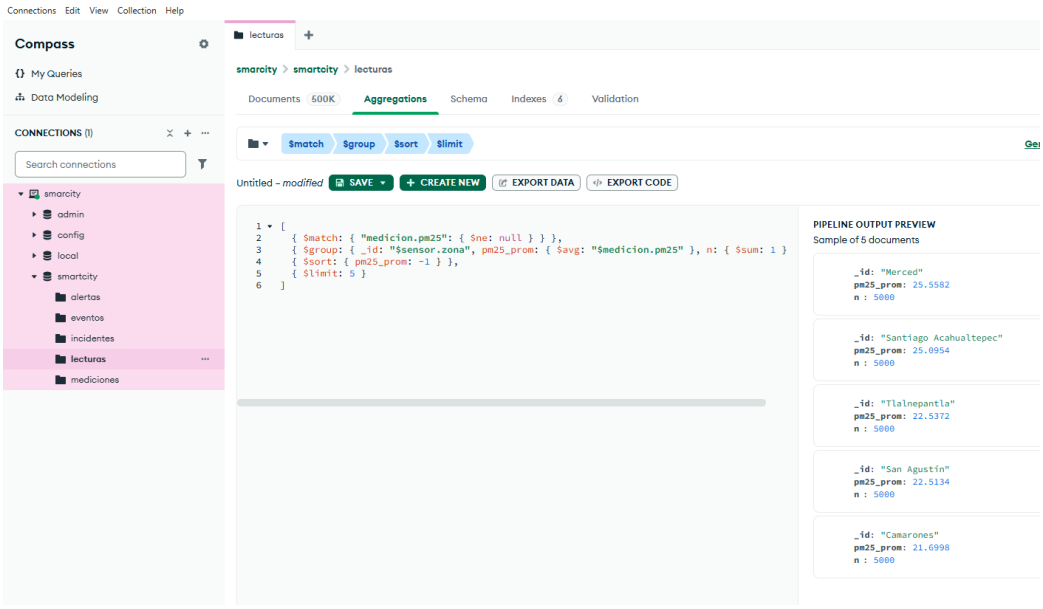


Figura 23: Aggregation Framework calculando el promedio de PM2.5 por zona.

B.3. Uso de índices en MongoDB (Explain)

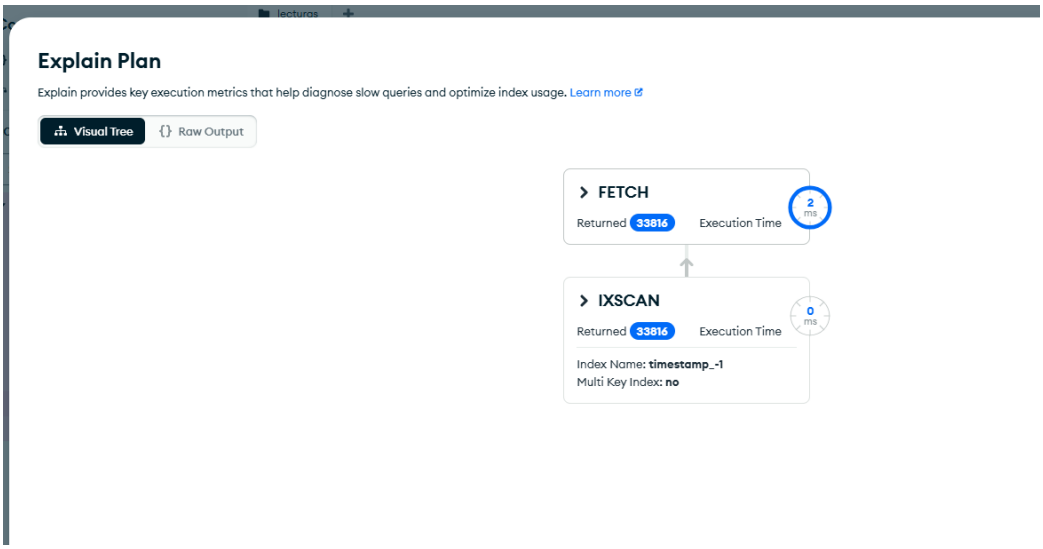


Figura 24: Plan de ejecución usando `IXSCAN` sobre el índice de `timestamp`.

C. Anexo C: Evidencia de Datos de Origen

C.1. Dataset de Tráfico (Kaggle)

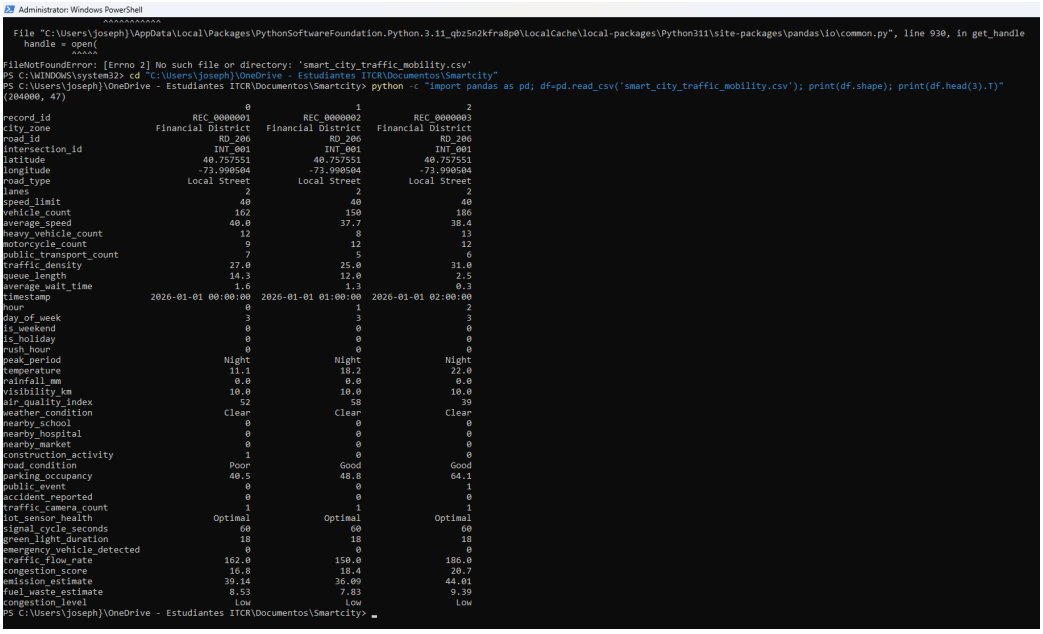


Figura 25: Inspección del CSV de tráfico (204,000 filas, 47 columnas).

C.2. Dataset de Calidad del Aire (OpenAQ)

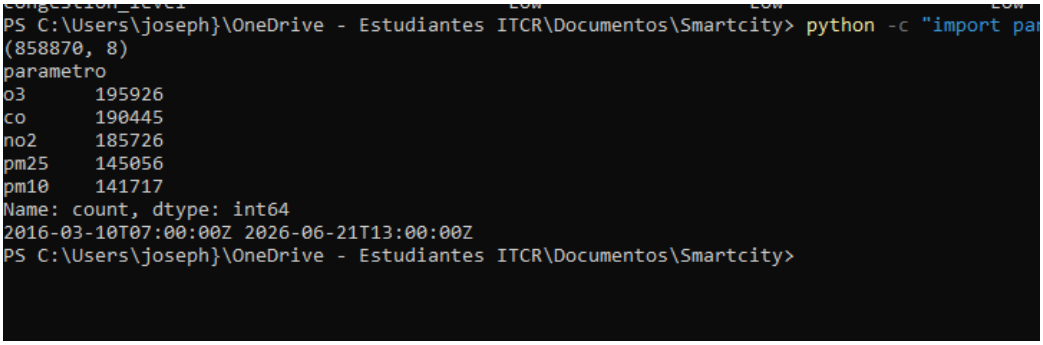


Figura 26: Inspección del CSV de calidad del aire (858,870 filas, 5 contaminantes).